

华中科技大学

课程实验报告

课程名称： 数据结构实验

专业班级 CS启明2401

学 号 U202414869

姓 名 韩轩

指导教师 王雄

报告日期 2024年5月29日

计算机科学与技术学院

目 录

1 基于顺序存储结构的线性表实现.....	1
1.1 问题描述	1
1.2 系统设计	1
1.3 系统实现	18
1.4 系统测试	19
1.5 实验小结	22
2 基于邻接表的图实现	23
2.1 问题描述	23
2.2 系统设计	23
2.3 系统实现	41
2.4 系统测试	42
2.5 实验小结	44
3 课程的收获和建议	45
3.1 基于顺序存储结构的线性表实现	45
3.2 基于邻接表的图实现.....	45
参考文献	46
附录 A 基于顺序存储结构的线性表实现	46
附录 B 基于链式存储结构的线性表实现	67
附录 C 基于二叉链表的二叉树实现	94
附录 D 基于邻接表的图实现.....	113

1 基于链式存储结构的线性表实现

1.1 问题描述

实验要求:

1. 实现对顺序表的基本操作;
2. 选择性实践对顺序表的进阶操作;
3. 设计演示系统。

通过实验达到:

1. 加深对线性表的概念、基本运算的理解;
2. 熟练掌握线性表的逻辑结构与物理结构的关系;
3. 物理结构采用顺序表, 熟练掌握线性表的基本运算的实现。

分析: 需要了解顺序表的结构, 同时演示系统要清晰明了。

1.2 系统设计

1.2.1 演示系统菜单的组织架构

演示系统分为基本操作和进阶操作两部分, 以一个菜单为主界面, 以序号 1 至 12 表示创建, 销毁, 清空, 插入, 删除, 求表长等基本操作, 以序号 13 至 18 表示保存为文件, 加载文件, 最大连续子数组和, 顺序表排序等进阶操作, 以序号 0 表示退出演示系统, 根据序号调用函数执行操作。

该演示系统将进阶操作与基础操作融合, 实现了多线性表的同时操作。

下图为演示系统菜单。

线性表数据结构实验菜单

- | | |
|---------|--------------|
| 1.创建线性表 | 10.插入元素 |
| 2.销毁线性表 | 11.删除元素 |
| 3.清空线性表 | 12.遍历线性表 |
| 4.线性表判空 | 13.最大连续子数组和 |
| 5.线性表长度 | 14.和为k的连续子数组 |
| 6.获取元素 | 15.线性表排序 |
| 7.定位元素 | 16.定位线性表 |
| 8.前驱元素 | 17.文件操作 |
| 9.后驱元素 | 18.遍历所有线性表 |
| 0.退出 | |

请选择你的操作：[0~18]:

1.2.2 ADT 数据结构设计

对链表数据类型的定义如下

```
typedef int status;
```

```
typedef int ElemType;
```

```
typedef struct{
```

```
    ElemType * elem;
```

```
    int length;
```

```
    int listsize;
```

```
}SqList;
```

```
\
```

```
typedef struct{
```

```
    struct { char name[30];
```

```
SqList L;  
  
    } elem[10];  
  
    int length;  
  
    int listsize;  
  
}LISTS;  
  
对一些常量的定义如  
  
#define FALSE 0  
  
#define OK 1  
  
#define ERROR 0  
  
#define INFEASIBLE -1  
  
#define OVERFLOW -2  
  
#define LIST_INIT_SIZE 100  
  
#define LISTINCREMENT 10
```

1.2.3 初始化表

输入: 顺序表 (未知状态)

输出: 函数执行状态

算法的思想描述: 为 L 结点分配存储空间, 置 $L.length = 0$, $L.listsize = LIST_INIT_SIZE$ 。

算法处理步骤:

- 1) 如果顺序表 L 不存在, 则为 L 分配存储空间。
- 2) 如果分配成功, 将置 $L.length = 0$, $L.listsize = LIST_INIT_SIZE$, 返回状态 OK。
- 3) 如果顺序表存在, 返回状态 INFEASIBLE 以指出无法创建。

时间复杂度: $O(1)$

空间复杂度: $O(1)$

代码实现:

`status InitList(SqList& L)`

// 线性表L不存在, 构造一个空的线性表, 返回OK, 否则返回INFEASIBLE。

```
{
    if (L.elem!=NULL) return INFEASIBLE;
    else {
        L.elem=(ElemType *) malloc(sizeof(ElemType)*LIST_INIT_SIZE);
        L.length=0; L.listsize=LIST_INIT_SIZE;
        return OK;
    }
}
```

1.2.4 销毁表

输入: 顺序表 (未知状态)

输出: 函数执行状态

算法的思想描述: 若 $L.elem$ 为 NULL, 返回 INFEASIBLE。否则将 L 销毁, 再将 L 置为 NULL, 将 $L.elem$ 和 $L.listsize$ 置为 0 , 返回 OK。

时间复杂度: $O(n)$

空间复杂度: $O(1)$

代码实现:

```
status DestroyList(SqList& L) {  
    if (L.elem == NULL) {  
        return INFEASIBLE;  
    } else {  
        free(L.elem);  
        L.elem = NULL;  
        L.length = 0;  
        L.listsize = 0;  
        return OK;  
    }  
}
```

1.2.5 清空表

输入: 顺序表 (未知状态)

输出: 函数执行状态

算法的思想描述: 若 L 为 NULL, 返回 INFEASIBLE。

否则删除 L 中所有元素, 置 L.length 为 0, 返回 OK。

时间复杂度: $O(n)$

空间复杂度: $O(1)$

代码实现:

```
status ClearList(SqList& L)  
// 如果线性表L存在, 删除线性表L中的所有元素, 返回OK, 否则返回  
// INFEASIBLE。  
{  
    if (L.elem==NULL) return INFEASIBLE;  
    else {  
        L.elem=NULL;  
        L.elem=(ElemType *)malloc(sizeof(ElemType));  
        L.length=0;  
    }
```

```
return OK;  
}
```

1.2.6 表判空

输入: 顺序表 (未知状态)

输出: 函数执行状态

算法的思想描述: 若 L 为 NULL, 返回 INFEASIBLE。否则判断 L.length 是否为 0, 若为 0, 则顺序表为空, 否则顺序表不为空。

时间复杂度: $O(1)$

空间复杂度: $O(1)$

代码实现:

```
status ListEmpty(SqList L)
```

```
// 如果线性表L存在, 判断线性表L是否为空, 空就返回TRUE, 否则返回FALSE;  
// 如果线性表L不存在, 返回INFEASIBLE。
```

```
{  
    if (L.elem==NULL) return INFEASIBLE;  
    else {  
        if (L.length==0) return OK;  
        else return ERROR;  
    }  
}
```

1.2.7 求表长

输入: 顺序表 L

输出: 顺序表的长度

算法处理步骤:

- 1) 如果顺序表 L 不存在, 则输出 INFEASIBLE。
- 2) 如果 L 存在, 返回 L.length 的值。

时间复杂度: $O(n)$

空间复杂度: $O(1)$

代码实现:

```
status ListLength(SqList L)
// 如果线性表L存在, 返回线性表L的长度, 否则返回INFEASIBLE。
{
    if (L.elem==NULL) return INFEASIBLE;
    else return L.length;
}
```

1.2.8 获取元素

输入: 顺序表 (未知状态)

输出: L 中第 i 个数据元素的值 e

算法处理步骤:

- 1) 如果顺序表 L 不存在, 则输出 INFEASIBLE。
- 2) 如果 L 存在, 如果 i 的输入不合法, 返回 ERROR。
- 3) 如果 i 的输入合法, 遍历获取要求的元素值, 并且存入 e 中。

代码实现:

```
status GetElem(SqList L,int i,ElemType &e)
// 如果线性表L存在, 获取线性表L的第i个元素, 保存在e中, 返回OK; 如果i不
// 合法, 返回ERROR; 如果线性表L不存在, 返回INFEASIBLE。
{
    if (L.elem==NULL) return INFEASIBLE;
    else {
        if ((i<1)||i>L.length) return ERROR;
        else {
            e=L.elem[i-1];
            return OK;
        }
    }
}
```

1.2.9 定位元素

输入: 顺序表 (未知状态)

输出:L 中第 1 个与 e 相等的元素的序号,若这样的数据元素不存在,则值为

0

算法的思想描述: 若 L.elem 为 NULL, 返回 INFEASIBLE。否则声明 i=0, 遍历顺序表查找是否有值与 e 相同的元素, 每进入下一个节点时 i 自增, 若找到相应元素则返回 i, 否则返回 0。

时间复杂度: $O(n)$

空间复杂度: $O(1)$

代码实现:

```
int LocateElem(SqList L, ElemType e)
```

```
// 如果线性表L存在, 查找元素e在线性表L中的位置序号并返回该序号; 如果e不存在, 返回0; 当线性表L不存在时, 返回INFEASIBLE (即-1)。
```

```
{  
    if (L.elem==NULL) return INFEASIBLE;  
    else {  
        for (int i=0;i<L.length;i++) {  
            if (L.elem[i]==e) return i+1;  
        }  
        return 0;  
    }  
}
```

1.2.10 获得前驱

输入: 顺序表 L, 元素 e, 引用参数 pre。

输出: 函数的执行状态。

算法思想的描述: 遍历顺序表。

算法处理步骤:

- 1) 如果顺序表 L 不存在, 则输出 INFEASIBLE。
- 2) 如果 L 存在, 从头开始遍历。
- 3) 若当前的后继值为 e, 则将当前的值赋给 pre, 输出 OK。
- 4) 若为找到, 则返回 INFEASIBLE。

时间复杂度: $O(n)$

空间复杂度: $O(1)$

代码实现:

```
status PriorElem(SqList L,ElemType e,ElemType &pre)
// 如果线性表L存在, 获取线性表L中元素e的前驱, 保存在pre中, 返回OK; 如
// 果没有前驱, 返回ERROR; 如果线性表L不存在, 返回INFEASIBLE。
{
    if (L.elem==NULL) return INFEASIBLE;
    else {
        for (int i=0;i<L.length-1;i++) {
            if (L.elem[i+1]==e) {
                pre=L.elem[i];
                return OK;
            }
        }
        return ERROR;
    }
}
```

1.2.11 求后继

输入: 顺序表 (未知状态)

输出: 函数执行状态

算法思想的描述:遍历顺序表.

算法处理步骤:

- 1) 如果顺序表 L 不存在, 则输出 INFEASIBLE。
- 2) 如果 L 存在, 从头开始遍历。
- 3) 若当前的前驱值为 e, 则将当前的值赋给 next, 输出 OK。
- 4) 若未找到, 则返回 ERROR。

时间复杂度: $O(n)$

空间复杂度: $O(1)$

代码实现:

```
status NextElem(SqList L,ElemType e,ElemType &next)
// 如果线性表L存在, 获取线性表L元素e的后继, 保存在next中, 返回OK; 如
// 果没有后继, 返回ERROR; 如果线性表L不存在, 返回INFEASIBLE。
{
    if (L.elem==NULL) return INFEASIBLE;
```

```
else {  
    for (int i=0;i<L.length-1;i++) {  
        if (L.elem[i]==e) {  
            next=L.elem[i+1];  
            return OK;  
        }  
    }  
    return ERROR;  
}  
}
```

1.2.12 插入元素

输入:顺序表 L, 插入位置 i, 插入元素 e。

输出: 函数的执行状态。

算法的思想描述: 查找元素和移动之后的点

算法处理步骤:

- 1) 如果顺序表 L 不存在, 则输出 INFEASIBLE。
- 2) 如果 L 存在, 判断 i 值是否符合要求, 不符合则返回 ERROR。
- 3) 如果 i 值合法, 将赋值点后的值后移。
- 4) 将插入位置的点赋值为 e。
- 5) 返回 OK。

时间复杂度: $O(n)$

空间复杂度: $O(1)$

代码实现:

```
status ListInsert(Sqlist &L,int i,ElemType e)
```

// 如果线性表L存在, 将元素e插入到线性表L的第i个元素之前, 返回OK; 当插入位置不正确时, 返回ERROR; 如果线性表L不存在, 返回INFEASIBLE。

```
{  
    if (L.elem==NULL) return INFEASIBLE;  
    else {  
        if ((i<1)|| (i>L.length+1)) return ERROR;  
        else {  
            L.length++;  

```

```
ElemType *newbase = (ElemType *)realloc(L.elem, (L.listsize + 1) *
sizeof(ElemType));
L.elem = newbase;
L.listsize++;
for (int j=L.length-1;j>=i-1;j--) {
    L.elem[j+1]=L.elem[j];
}
L.elem[i-1]=e;
return OK;
}
}
}
```

1.2.13 删除元素

输入: 顺序表 (未知状态)

输出: 函数执行状态

算法的思想描述: 若 L 为 NULL, 返回 INFEASIBLE。若 i 输入不合法则返回 ERROR。否则删除指定元素, 将其存至 e, 将之后的值一次前移, 返回 OK。

时间复杂度: $O(n)$

空间复杂度: $O(1)$

代码实现:

```
status ListDelete(SqList &L,int i,ElemType &e)
```

// 如果线性表L存在, 删除线性表L的第i个元素, 并保存在e中, 返回OK; 当删除位置不正确时, 返回ERROR; 如果线性表L不存在, 返回INFEASIBLE。

```
{
    if (L.elem==NULL) return INFEASIBLE;
    else {
        if ((i<1)|| (i>L.length-1)) return ERROR;
        else {
            e=L.elem[i-1];
            for (int j=i;j<L.length;j++) {
                L.elem[j-1]=L.elem[j];
            }
        }
    }
}
```

```
}  
L.length--;  
return OK;  
}  
}
```

1.2.14 遍历顺序表

输入:顺序表 L.

输出: 函数的执行状态

算法思想描述: 遍历顺序表并输出每一个元素的值.

- 1) 如果顺序表 L 不存在, 则输出 INFEASIBLE。
- 2) 如果 L 存在, 从头开始遍历并输出。
- 3) 返回 OK。

时间的复杂度: $O(n)$.

空间的复杂度: $O(1)$.

代码实现:

status ListTraverse(SqList L)

// 如果线性表L存在, 依次显示线性表中的元素, 每个元素间空一格, 返回OK;
如果线性表L不存在, 返回INFEASIBLE。

```
{  
    if (L.elem==NULL) return 0;  
    else {  
        for(int i=0;i<L.length;i++) {  
            printf("%d",L.elem[i]);  
            if(i<L.length-1) printf(" ");  
        }  
        return OK;  
    }  
}
```

1.2.15 顺序表排序

输入:顺序表 L

输出: 函数执行状态

算法处理的步骤:

- 1) 如果 L 不存在, 输出INFEASIBLE。
- 2) 对 L进行冒泡排序
- 3) 输出OK

时间复杂度: $O(n)$

空间复杂度: $O(n)$

代码实现;

```
status SortList(SqList& L) {  
    if (L.elem == NULL)  
        return INFEASIBLE;  
    else {  
        for (int i = 0; i < L.length - 1; i++) {  
            for (int j = 0; j < L.length - 1 - i; j++) {  
                int term;  
                if (L.elem[j] > L.elem[j + 1]) {  
                    term = L.elem[j];  
                    L.elem[j] = L.elem[j + 1];  
                    L.elem[j + 1] = term;  
                }  
            }  
        }  
        return OK;  
    }  
}
```

以下为算法的流程图:



1.2.16 最大连续子数组和

输入: 顺序表 (未知状态)

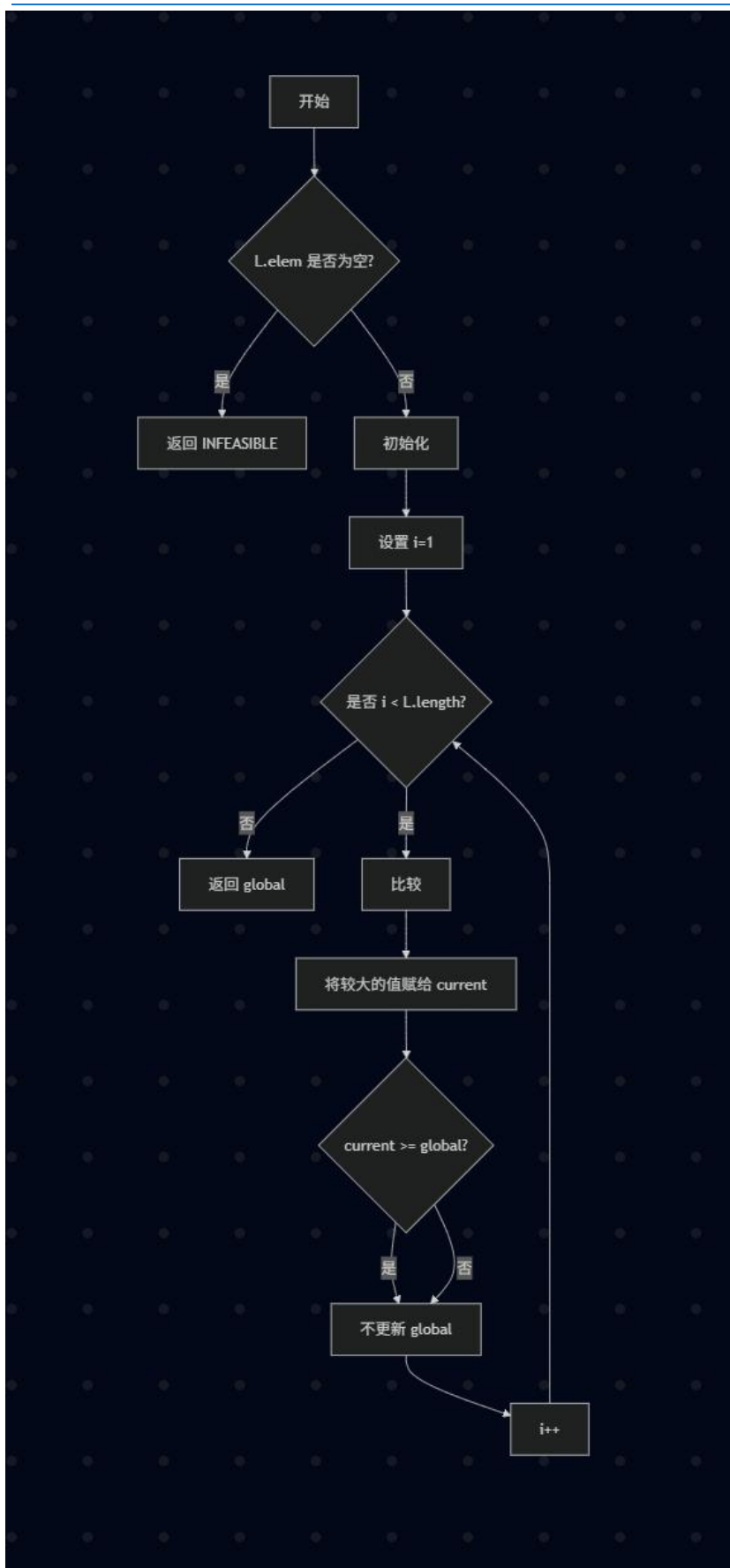
输出: 函数执行状态

算法的思想描述: 该算法运用了动态规划思想来求解最大子数组和问题。它维护两个变量, 一个记录当前子数组的最大和, 另一个记录全局最大子数组和。从数组第二个元素开始遍历, 每次判断当前元素与当前子数组和加上当前元素后的较大值作为新的当前子数组和, 若当前子数组和大于全局最大子数组和则更新全局最大值, 最终得到整个数组的最大子数组和。

代码实现:

```
status MaxSubArray(SqList& L) {  
    if (L.elem==NULL) return INFEASIBLE;  
    else {  
        int current=L.elem[0], global=L.elem[0];  
        for (int i=1; i<L.length; i++) {  
  
current=((current+L.elem[i]>L.elem[i])?(current+L.elem[i]):(L.elem[i]))  
);  
            if (current >= global) {  
                global=current;  
            }  
        }  
        return global;  
    }  
}
```

以下为算法的流程图



1.2.17 子数组和为 k

输入: 顺序表 (未知状态), 所求 k

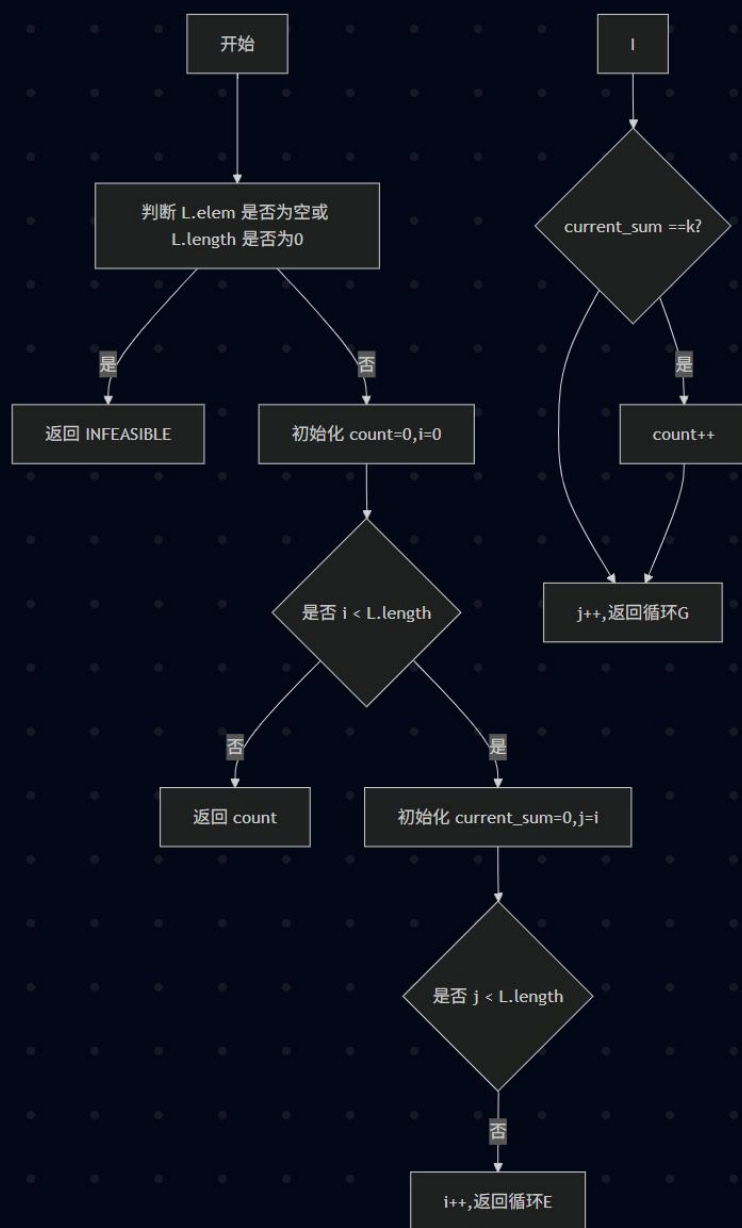
输出: 和为 k 的子数组数

算法的思想描述: 该算法采用暴力枚举的方式, 通过两层循环遍历数组, 外层循环确定子数组的起始位置, 内层循环从起始位置开始逐步累加元素, 计算连续子数组的和, 每当累加和等于给定值k时, 计数器加1, 最终返回满足条件的连续子数组的个数。

代码实现:

```
status SubArrayNum(SqList& L, int k) {  
    if (L.elem == NULL || L.length == 0) return INFEASIBLE;  
  
    int count = 0;  
  
    for (int i = 0; i < L.length; i++) {  
        int current_sum = 0;  
  
        for (int j = i; j < L.length; ++j) {  
            current_sum += L.elem[j];  
  
            if (current_sum == k) {  
                count++;  
            }  
        }  
    }  
  
    return count;  
}
```

以下为算法的流程图



1.2.18 文件操作

输入: 顺序表 (未知状态), 文件名

输出: 函数执行状态

时间复杂度: $O(n)$

空间复杂度: $O(1)$

代码实现:

```
status SaveList(SqList L, char FileName[]) {
    if (L.elem == NULL) return INFEASIBLE;
    FILE *fp = fopen(FileName, "w");
    if (fp == NULL) return ERROR;

    fprintf(fp, "%d ", L.length);
    for (int i = 0; i < L.length; i++) {
        fprintf(fp, "%d ", L.elem[i]);
    }

    fclose(fp);
    return OK;
}

status LoadList(SqList &L, char FileName[]) {
    if (L.elem != NULL) return INFEASIBLE;
    FILE *fp = fopen(FileName, "r");
    if (fp == NULL) return ERROR;

    int length;
    if (fscanf(fp, "%d", &length) != 1) {
        fclose(fp);
        return ERROR;
    }

    L.elem = (ElemType *)malloc(length * sizeof(ElemType));
    if (L.elem == NULL) {
        fclose(fp);
```

```
    return ERROR;
}

for (int i = 0; i < length; i++) {
    if (fscanf(fp, "%d", &L.elem[i]) != 1) {
        free(L.elem);
        L.elem = NULL;
        fclose(fp);
        return ERROR;
    }
}

L.length = length;
L.listsize = length;

fclose(fp);
return OK;
}
```

1.3 系统实现

1.3.1 程序开发环境与语言

PC 操作系统为 windows 操作系统使用语言为 C++。

1.3.2 代码的组织结构

演示系统以一个菜单作为交互界面，用户通过输入命令对应的编号来调用相应的函数来实现创建表，销毁表，清空表，插入元素，删除元素，求表长，判空表，求前驱，求后继，遍历链表等基本操作，以及保存为文件，加载文件，翻转链表，对链表排序等进阶操作。

程序主函数为一个 switch 结构，根据输入的数字，执行不同的语句，进而调用不同的函数，宏定义函数返回值 ERROR 为 0，INFEASIBLE 为-1，OVERFLOW 为-2，OK 为 1。

1.4 系统测试

程序开发及实现环境：Win11 下使用VSCode进行编译和调试，开发语言为 C 语言。

下表为正常样例测试的输入，预期结果与实际输出。

函数	输入	实际输出	预期结果
初始化表	1	线性表创建成功	线性表创建成功
销毁表	2	线性表销毁成功	线性表销毁成功
销毁表	2	线性表不存在	线性表不存在
初始化表	1	线性表创建成功	线性表创建成功
线性表判空	4	线性表为空	线性表为空
插入元素	10 1 1	插入成功	插入成功
插入元素	10 2 5	插入成功	插入成功
插入元素	10 3 7	插入成功	插入成功
求表长	5	线性表长度为 3	线性表长度为 3

华中科技大学课程实验报告

获取元素	6 2	获取元素为 5	获取元素为 5
查找元素	7 7	元素位置为 3	元素位置为 3
查找前驱	8 5	元素的前驱为 1	元素的前驱为 1
查找后继	9 5	元素的后继为 7	元素的后继为 7
遍历	12	1 5 7	1 5 7
删除元素	11 2	删除元素 5	删除元素 5
保存文件	13 123	保存文件成功	保存文件成功
销毁表	2	线性表销毁成功	线性表销毁成功
读取文件	17 F " :/shiyan	读入文件成功	读入文件成功
遍历链表	12	1 7	1 7

1.5 实验小结

本次实验让我对基于顺序存储结构的线性表的了解更进一步。

演示系统的搭建，让我体会到了主函数和子函数的关系，以及如何搭建一个可以调用不同模块的系统。

在编写顺序表的插入和删除乃至扩容等函数时，如何精确地操作数组元素和长度信息是一大难点，比如在插入新元素时，必须先判断存储空间是否足够，再将插入位置及之后的元素后移，然后才能将新元素放入指定位置，同时更新表长。

经过这次实验，我明白了操作顺序对程序正确性的关键作用。总的来说，本次数据结构实验提高了我的编程能力，让我对系统整体设计有了更深的认识。

2 基于邻接表的图实现

2.1 问题描述

实验要求：

1. 实现对图的基本操作；
2. 选择性实践对图的进阶操作；
3. 设计演示系统。

通过实验达到：

1. 加深对图的概念、基本运算的理解；
2. 熟练掌握图的逻辑结构与物理结构的关系；
3. 以邻接表作为物理结构，熟练掌握图基本运算的实现。

分析：需要了解图的结构，同时演示系统要清晰明了。

2.2 系统设计

2.2.1 演示系统菜单的组织架构

演示系统分为基本操作和进阶操作两部分，以一个菜单为主界面，以序号 1 至 12 表示创建，销毁，清空，插入，删除，遍历等基本操作，以序号 13 至 17 表示保存为文件，求最短通路，求距某顶点距离小于 d 的顶点，对顶点进行修改等进阶操作，以序号 0 表示退出演示系统，根据序号调用函数执行操作。

2.2.2 ADT 数据结构设计

对图数据类型的定义如下

```
typedef int status;

typedef int KeyType;

typedef enum {DG, DN, UDG, UDN} GraphKind;

typedef struct {

    KeyType key;
```

```
char others[20];

} VertexType; //顶点类型定义

typedef struct ArcNode {           //表结点类型定义
    int adjvex;                    //顶点位置编号
    struct ArcNode *nextarc;       //下一个表结点指针
} ArcNode;

typedef struct VNode{              //头结点及其数组类型定义
    VertexType data;               //顶点信息
    ArcNode *firstarc;             //指向第一条弧
} VNode, AdjList[MAX_VERTEX_NUM];

typedef struct { //邻接表的类型定义
    AdjList vertices;              //头结点数组
    int vexnum, arcnum;            //顶点数、弧数
    GraphKind kind;                //图的类型
} ALGraph;
```

对一些常量的定义如下

```
#define TRUE 1
#define FALSE 0
#define OK 1
#define ERROR 0
#define INFEASIBLE -1
#define OVERFLOW -2
#define MAX_VERTEX_NUM 20
```

2.2.3 创建图

输入: 图 G (未知状态), 顶点集 $V[]$, 边集 $VR[]$

输出: 函数执行状态

算法的思想描述: 该算法用于根据给定的顶点数组和边数组构造一个无向图的邻接表结构。首先遍历顶点数组, 检查每个顶点的关键字和 `others` 字段是否有重复, 如有重复则返回错误; 若合法, 则将顶点信息存入图的顶点数组中并统计顶点数。接着检查顶点数是否超出最大限制或为零, 若不符合要求则返回错误。然后处理边数组, 对于每条边, 分别查找其对应的起点和终点在顶点数组中的位置, 若找不到则返回错误; 找到后, 分别为起点和终点创建弧节点并插入到对应的邻接表中, 同时统计边数。最后设置图的类型为无向图并返回成功状态。

时间复杂度: $O(n)$

空间复杂度: $O(n)$

代码实现:

```
int same(char a[],char b[]){
    int i=0;
    int sign=1;
    for(;a[i]!='\0' && b[i]!='\0';i++){
        if(a[i]!=b[i]){
            sign=0;
            break;
        }
    }
    if(a[i]!='\0' || b[i]!='\0') sign=0;
    return sign;
}
```

```
}
```

```
status CreateCraph(ALGraph &G,VertexType V[],KeyType VR[][2])
```

```
/*根据V和VR构造图T并返回OK，如果V和VR不正确，返回ERROR
```

```
如果有相同的关键字，返回ERROR。此题允许通过增加其它函数辅助实现本关任务*/
```

```
{
```

```
int i=0,j=0;    //当前头结点、边个数
```

```
for(;V[i].key!=-1;i++){
```

```
    for(int n=0;n<i;n++){
```

```
        if(V[n].key==V[i].key || same(V[n].others,V[i].others)){
```

```
            return ERROR;
```

```
        }
```

```
    }
```

```
    G.vertices[i].data=V[i];
```

```
    G.vertices[i].firstarc=NULL;
```

```
}
```

```
G.vexnum=i;
```

```
if(G.vexnum > MAX_VERTEX_NUM) return ERROR;
```

```
if(G.vexnum == 0) return ERROR;
```

```
int tail,head;
```

```
for(;VR[j][0]!=-1;j++){
```

```
    tail=head=-1;
```

```
    for(int m=0;m<G.vexnum;m++){
```

```
        if(G.vertices[m].data.key==VR[j][0]){
```

```
            tail=m;
```

```
        }
```

```
        if(G.vertices[m].data.key==VR[j][1]){
```

```
            head=m;
```

```
        }
```

```
    }
```

```
if(tail==-1 || head==-1) return ERROR; //找到弧尾与弧头
```

```
ArcNode *p=(ArcNode *)malloc(sizeof(ArcNode));
```

```
p->adjvex=head;
p->nextarc=G.vertices[tail].firstarc;
G.vertices[tail].firstarc=p;

ArcNode *q=(ArcNode *)malloc(sizeof(ArcNode));
q->adjvex=tail;
q->nextarc=G.vertices[head].firstarc;
G.vertices[head].firstarc=q;
}
G.arcnum=j;
G.kind=UDG;
return OK;
}
```

2.2.4 销毁图

输入: 图 G

输出: 函数执行状态

算法的思想描述:遍历所有弧结点

算法处理步骤:

- 1) 用两个临时变量分别记录首节点与下一结点。
- 2) 删除首个顶点然后两个结点开始移动, 遍历完一个顶点的所有邻接点。
- 3) 重复直至所有结点都遍历过。
- 4) 删除成功, 返回 OK。

时间复杂度: $O(n)$

空间复杂度: $O(1)$

代码实现:

```
status DestroyGraph(ALGraph &G)
/*销毁无向图G,删除G的全部顶点和边*/
{
    for(int i=0; i<G.vexnum; i++) {
        ArcNode *p=G.vertices[i].firstarc;
        for(; p!=NULL;) {
            ArcNode *p1=p;
            p=p->nextarc;
```

```
    free(p1);
}
G.vertices[i].firstarc=NULL;
}
G.vexnum=0;
G.arcnum=0;
return OK;
}
```

2.2.5 定位顶点

输入: 图 G, 要查找顶点的关键字

输出: 要查找顶点的位序

算法的思想描述: 用 for 循环遍历邻接表, 如果当前顶点关键字与所找关键字相等, 则返回当前顶点序号。如果没找到, 返回-1。

时间复杂度: $O(n)$

空间复杂度: $O(1)$

代码实现:

```
int LocateVex(ALGraph G,KeyType u)
//根据u在图G中查找顶点, 查找成功返回位序, 否则返回-1;
{
    for(int i=0; i<G.vexnum; i++) {
        if(u == G.vertices[i].data.key) return i;
    }
    return -1;
}
```

2.2.6 修改顶点

输入: 图 G, 要修改结点的关键字, 以及要修改成的值 value

输出: 函数执行状态

算法的思想描述: 寻找赋值顶点并赋值

算法处理步骤:

- 1) 定位需要赋值的顶点的位置。
- 2) 将该顶点的关键字和值域重新赋值。
- 3) 修改成功，返回 OK。

时间复杂度: $O(n)$

空间复杂度: $O(1)$

代码实现:

```
status PutVex(ALGraph &G,KeyType u,VertexType value)
//根据u在图G中查找顶点，查找成功将该顶点值修改成value，返回OK；
//如果查找失败或关键字不唯一，返回ERROR
{
    int location= -1;
    for(int i=0; i<G.vexnum; i++) {
        if(G.vertices[i].data.key == u){
            G.vertices[i].data=value;
            for(int j=0; j<i; j++) {
                if(G.vertices[i].data.key==G.vertices[j].data.key) return ERROR;
            }
            return OK;
        }
    }
    return ERROR;
}
```

2.2.7 第一邻接点

输入: 图 G, 要查找邻接点的顶点的关键字

输出: 第一邻接点的位序

算法思想描述: 调用定位函数查找关键字为 u 的结点。如果 $i == G.vexnum \parallel G.vertices[i].firstarc == NULL$, 即没找到该点或该点无邻接点, 则返回-1。否则返回 $G.vertices[i].firstarc \rightarrow adjvex$ 。

算法处理步骤:

- 1) 定位关键字为 v 的顶点的位置。
- 2) 若下一个弧结点不为空则返回它否则返回-1。

时间复杂度: $O(n)$

空间复杂度: $O(1)$

代码实现:

```
int FirstAdjVex(ALGraph G,KeyType u)
```

```
//根据u在图G中查找顶点，查找成功返回顶点u的第一邻接顶点位序，否则返回-1;
```

```
{
```

```
    for(int i=0; i<G.vexnum; i++) {
```

```
        if(G.vertices[i].data.key == u) return G.vertices[i].firstarc->adjvex;
```

```
    }
```

```
    return -1;
```

```
}
```

2.2.8 下一邻接点

输入: 图 G, 要查找邻接点的顶点的关键字

输出: 下一邻接点的位序

算法的思想描述:调用定位函数查找关键字为 u 的结点。如果 $i == G.vexnum$, 返回 ERROR。用 while 循环遍历该顶点的所有邻接点, 找到另一个顶点, 用 p 指向该邻接点。如果 $p == NULL \parallel p \rightarrow nextarc == NULL$, 即 v 与 w 不相邻, 或 v 相对 w 无下一邻接点, 返回-1。否则返回 $return p \rightarrow nextarc \rightarrow adjvex$ 。

时间复杂度: $O(n)$

空间复杂度: $O(1)$

代码实现:

```
int NextAdjVex(ALGraph G,KeyType v,KeyType w)
```

//v对应G的一个顶点,w对应v的邻接顶点;操作结果是返回v的(相对于w)下一个邻接顶点的位序;如果w是最后一个邻接顶点,或v、w对应顶点不存在,则返回-1。

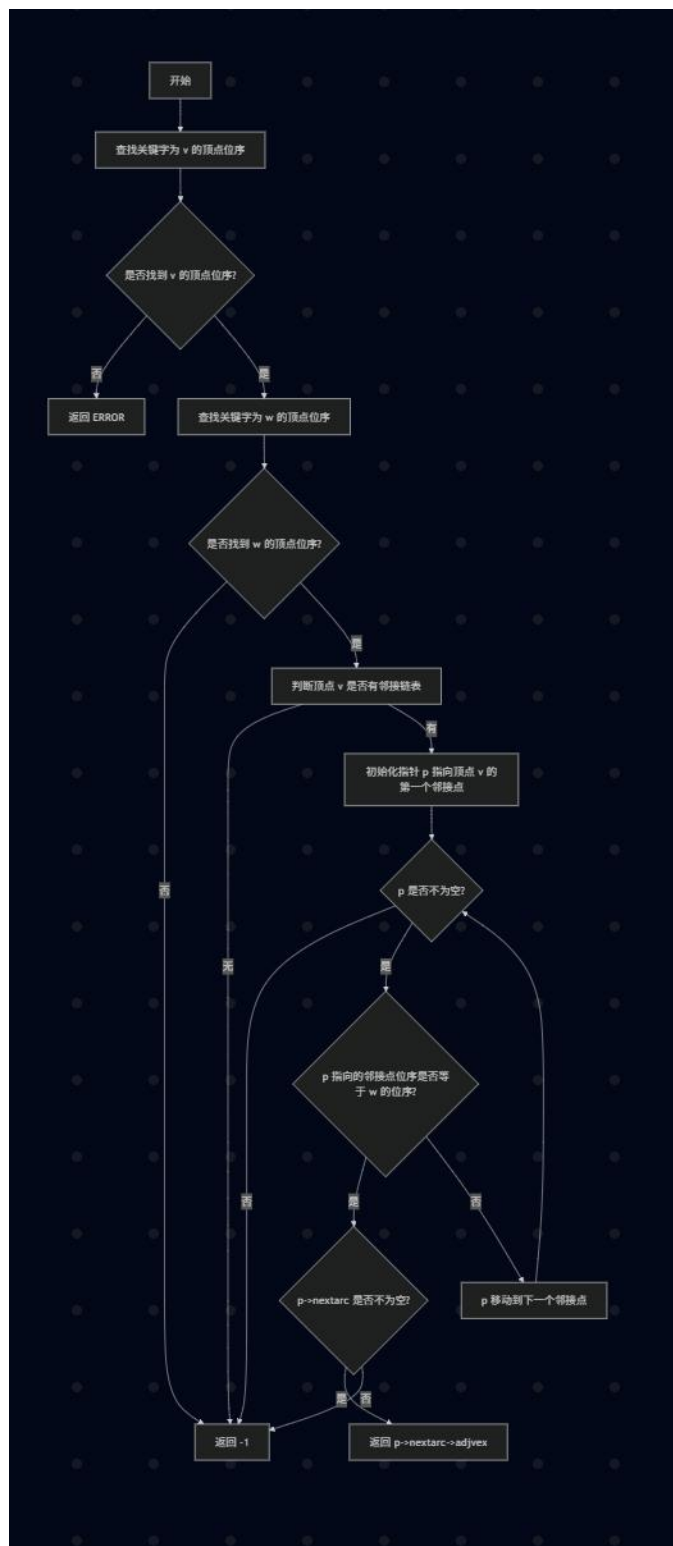
```
{
    int signv=-1,signw=-1;
    for(int i=0;i<G.vexnum;i++){
        if(G.vertices[i].data.key==w){
            signw=i;
        }
        if(G.vertices[i].data.key==v){
            signv=i;
        }
    }
    if(signw==-1) return -1;
    else{
        if(G.vertices[signv].firstarc==NULL) return -1;
        ArcNode *p=G.vertices[signv].firstarc;
        while(p!=NULL){
            if(p->adjvex==signw){
                if(p->nextarc != NULL) return p->nextarc->adjvex;
            } else return -1;
        }
    }
}
```

```

p=p->nextarc;
}
return -1;
}
}

```

以下为算法的流程图：



2.2.9 插入顶点

输入: 图 G, 要插入顶点的关键字

输出: 函数执行状态

算法的思想描述: 如果 $G.vexnum == MAXVERTEXNUM$, 返回 ERROR。
如 若不然使用 LocateVex 查找要插入结点的关键字。如果要插入顶点的关键字已出 现, 则返回 ERROR。否则插入新顶点, 更新顶点数, 返回 OK。

时间复杂度: $O(n)$

空间复杂度: $O(1)$

代码实现:

```
status InsertVex(ALGraph &G, VertexType v)
//在图G中插入顶点v, 成功返回OK, 否则返回ERROR
{
    //默认原先关键字不重复
    if(G.vexnum+1 > MAX_VERTEX_NUM) return ERROR;

    for(int i=0; i<G.vexnum; i++) {
        if(G.vertices[i].data.key == v.key) {
            return ERROR;
        }
    }

    G.vertices[G.vexnum].data=v;
    G.vertices[G.vexnum].firstarc=NULL;
    G.vexnum++;

    return OK;
}
```

2.2.10 删除顶点

输入: 图 G, 要删除的顶点的关键字

输出: 函数的执行状态.

算法思想的描述: 寻找所要删除的顶点, 记录它所连接的所有弧结点, 在其他顶点中将所有的弧结点删除

时间复杂度: $O(n*n)$

空间复杂度: $O(1)$

代码实现:

```
status InsertVex(ALGraph &G,VertexType v)
//在图G中插入顶点v, 成功返回OK,否则返回ERROR
{
    //默认原先关键字不重复
    if(G.vexnum+1 > MAX_VERTEX_NUM) return ERROR;

    for(int i=0; i<G.vexnum; i++) {
        if(G.vertices[i].data.key == v.key) {
            return ERROR;
        }
    }

    G.vertices[G.vexnum].data=v;
    G.vertices[G.vexnum].firstarc=NULL;
    G.vexnum++;

    return OK;
}
```

2.2.11 插入弧

输入: 图 G, 和顶点关键字类型相同的给定值 v,w

输出: 函数执行状态

算法的思想描述: 查找 v,w 的关键字, 如果其中任意一个未找到, 则返回 ERROR, 否则分别在这两个关键字后的链表中添加相应弧, 返回 OK。算法处理步骤:

- 1) 寻找所要插入的结点 v,w
- 2) 在 v 和 w 中分别用首插法

3) 总弧数加 1。

时间复杂度: $O(n)$

空间复杂度: $O(1)$

代码实现:

```
status InsertArc(ALGraph &G,KeyType v,KeyType w)
//在图G中增加弧<v,w>, 成功返回OK,否则返回ERROR
{
    int a=-1, b=-1;
    for(int i=0; i<G.vexnum; i++) {
        if(v == G.vertices[i].data.key) a=i;
        if(w == G.vertices[i].data.key) b=i;
    }

    if((a==-1)||(b==-1)) return ERROR;

    ArcNode *p= G.vertices[a].firstarc;
    for(; p!= NULL; p=p->nextarc) {
        if(p->adjvex==b) return ERROR;
    }
    p= G.vertices[b].firstarc;
    for(; p!= NULL; p=p->nextarc) {
        if(p->adjvex==a) return ERROR;
    }

    ArcNode *newArc = (ArcNode *)malloc (sizeof(ArcNode));
    newArc->adjvex = b; newArc->nextarc = G.vertices[a].firstarc;
    G.vertices[a].firstarc = newArc;

    newArc = (ArcNode *)malloc (sizeof(ArcNode));
    newArc->adjvex = a; newArc->nextarc = G.vertices[b].firstarc;
    G.vertices[b].firstarc = newArc;

    G.arcnum++;
}
```

```
for (int i = 0; i < G.vexnum; i++) {
    ArcNode *p = G.vertices[i].firstarc;
    while (p != NULL) {
        ArcNode *p1 = G.vertices[i].firstarc;
        while (p1 != p) {
            if (p1->adjvex == p->adjvex) {
                return ERROR;
            }
            p1 = p1->nextarc;
        }
        p = p->nextarc;
    }
}

return OK;
}
```

2.2.12 删除弧

输入: 图 G , 和顶点关键字类型相同的给定值 v, w

输出: 函数的执行状态。

算法的思想描述: 调用函数查找 v, w 的关键字, 如果其中任意一个未找到, 则返回 ERROR, 否则分别在这两个关键字后的链表中寻找并删除对应结点的弧, 返回 OK。

算法处理步骤:

- 1) 寻找所要删除的弧的信息。
- 2) 分别在 v 中删除 w 和在 w 中删除 v 。
- 3) 若删除成功则返回 OK。

时间复杂度: $O(n)$

空间复杂度: $O(1)$

代码实现:

```
status DeleteArc(ALGraph &G, KeyType v, KeyType w)
//在图G中删除弧<v,w>, 成功返回OK,否则返回ERROR
{
    int a = -1, b = -1;
    for (int i = 0; i < G.vexnum; i++) {
        if (G.vertices[i].data.key == v) a = i;
        if (G.vertices[i].data.key == w) b = i;
    }
    if (a == -1 || b == -1) return ERROR;

    ArcNode **p = &G.vertices[a].firstarc;
    int flag = 0;
    for (; (*p) != NULL; p = &(*p)->nextarc) {
        if ((*p)->adjvex == b) {
            ArcNode *temp = *p;
            *p = temp->nextarc;
            free(temp);
            flag = 1;
            break;
        }
    }
}
```



```
    }  
}  
if (flag == 0) return ERROR;  
  
p = &G.vertices[b].firstarc;  
flag = 0;  
for (; (*p) != NULL; p = &(*p)->nextarc) {  
    if ((*p)->adjvex == a) {  
        ArcNode *temp = *p;  
        *p = temp->nextarc;  
        free(temp);  
        flag = 1;  
        break;  
    }  
}  
if (flag == 0) return ERROR;  
  
G.arcnum--;  
return OK;  
}
```

2.2.13 深度优先遍历

输入: 图 G

输出: 函数执行状态

算法的思想描述: 如果 $G.vexnum == 0$, 即图不存在, 返回 INFEASIBLE。定义标记数组用于标记顶点是否已访问; 递归地从第一个顶点开始访问其余顶点, 每次访问时更新标记数组, 以确保每个节点只被访问一次; 返回 OK。。算法处理步骤:

- 1) 构造一个对一个顶点所有邻接结点的递归函数 `dfs`
- 2) 依次读取每个顶点并用标记数组记录已读取的顶点
- 3) 用 `visit` 函数将深度遍历的结点信息逐个打印。

时间复杂度: $O(n)$

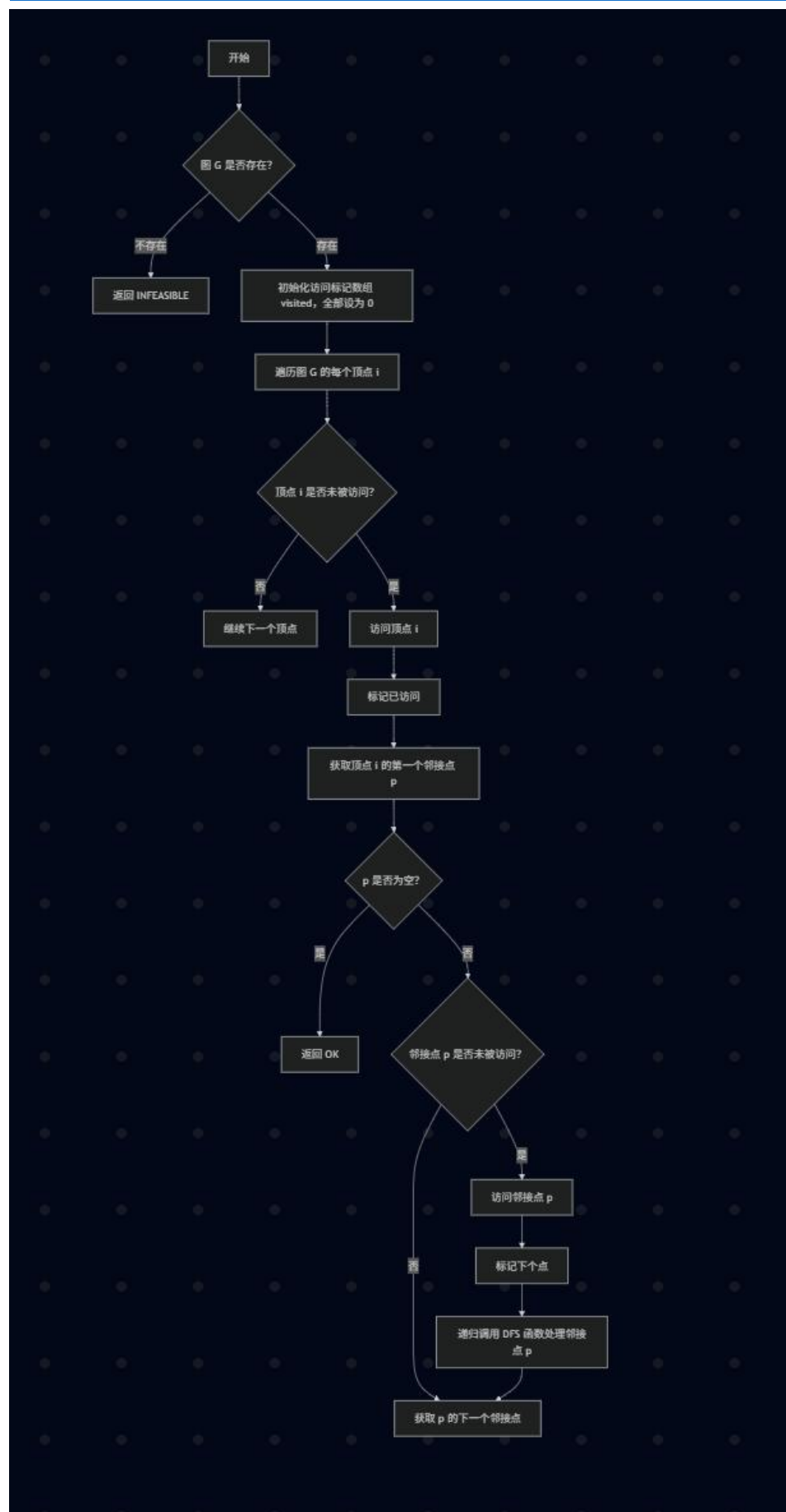
空间复杂度: $O(1)$

代码实现:

```
void DFS(ALGraph &G, int i, int visited[], void (*visit)(VertexType)) {
    ArcNode *p = G.vertices[i].firstarc;
    while (p != NULL) {
        if (!visited[p->adjvex]) {
            visit(G.vertices[p->adjvex].data);
            visited[p->adjvex] = 1;
            DFS(G, p->adjvex, visited, visit);
        }
        p = p->nextarc;
    }
}
```

```
status DFSTraverse(ALGraph &G, void (*visit)(VertexType))
{
    int visited[MAX_VERTEX_NUM] = {0};
    for (int i = 0; i < G.vexnum; i++) {
        if (!visited[i]) {
            visit(G.vertices[i].data);
            visited[i] = 1;
            DFS(G, i, visited, visit);
        }
    }
    return OK;
}
```

以下为算法流程图:



2.2.14 广度优先遍历

输入: 图 G

输出: 函数的执行状态

算法思想描述: 利用队列这一数据结构, 保存每一次广搜的所有弧结点并将他们在下一次广搜时访问所有未被读取的首节点。

时间的复杂度: $O(n)$.

空间的复杂度: $O(n)$.

代码实现:

```
void BFS(ALGraph &G, int start, int visited[], void
(*visit)(VertexType)) {
    int queue[MAX_VERTEX_NUM];
    int front = 0, rear = 0;
    queue[rear++] = start;

    while (front < rear) {
        int i = queue[front++];
        ArcNode *p = G.vertices[i].firstarc;
        while (p != NULL) {
            if (!visited[p->adjvex]) {
                visit(G.vertices[p->adjvex].data);
                visited[p->adjvex] = 1;
                queue[rear++] = p->adjvex;
            }
            p = p->nextarc;
        }
    }
}
```

```
}
```

```
status BFSTraverse(ALGraph &G,void (*visit)(VertexType))
```

```
{
```

```
    int visited[MAX_VERTEX_NUM] = {0};
```

```
    for (int i = 0; i < G.vexnum; i++) {
```

```
        if (!visited[i]) {
```

```
            visit(G.vertices[i].data);
```

```
            visited[i] = 1;
```

```
            BFS(G, i, visited, visit);
```

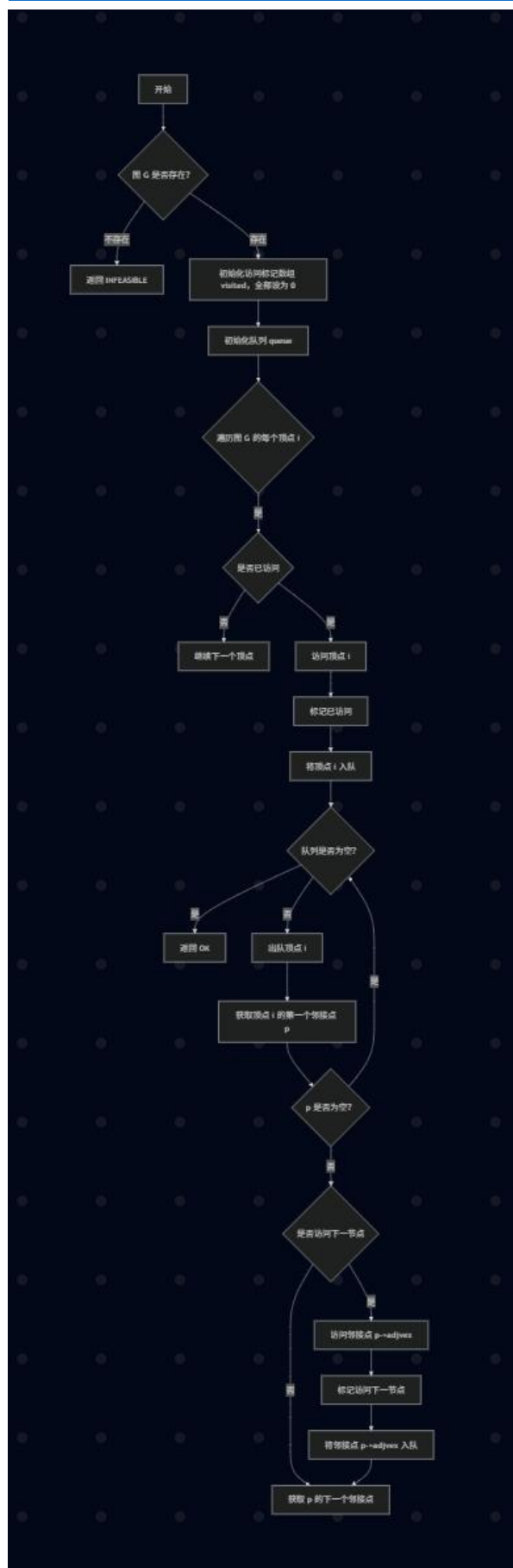
```
        }
```

```
    }
```

```
    return OK;
```

```
}
```

以下为算法流程图：



2.2.15 连通分支数

输入: 图 G

输出: 连通分量的个数

算法思想描述: 利用深度优先搜索 (DFS) 来找出无向图中的连通分量个数。

时间的复杂度: $O(n)$.

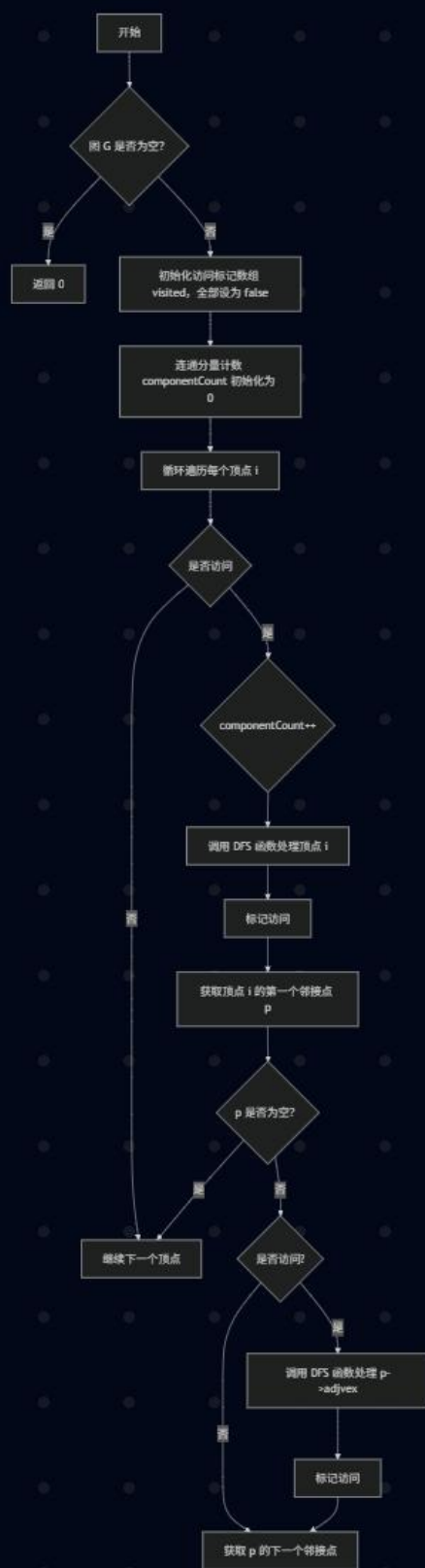
空间的复杂度: $O(n)$.

代码实现:

```
int ConnectedComponentsNums (ALGraph &G) {  
    if (G.vexnum == 0) {  
        return 0; // 如果图中没有顶点，连通分量为 0  
    }  
  
    std::vector<bool> visited(G.vexnum, false); // 记录顶点是否访问过  
    int componentCount = 0; // 连通分量计数  
  
    // 深度优先搜索辅助函数  
    auto DFS = [&](int v, auto &DFS) -> void {  
        visited[v] = true;  
        ArcNode *p = G.vertices[v].firstarc;  
        while (p != NULL) {  
            if (!visited[p->adjvex]) {  
                DFS(p->adjvex, DFS);  
            }  
            p = p->nextarc;  
        }  
    }  
}
```

```
    }  
};  
  
// 遍历所有顶点，查找未访问的顶点并进行 DFS  
for (int i = 0; i < G.vexnum; i++) {  
    if (!visited[i]) {  
        componentCount++; // 每次发现一个未访问的顶点，连通分量加  
1  
        DFS(i, DFS);  
    }  
}  
  
return componentCount;  
}
```

以下为算法流程图



2.2.15 连通分支数

输入：图 G 、顶点 v 和顶点 w

输出：顶点 v 到顶点 w 的最短路径长度，若不存在路径则返回 -1 ，若起点和终点相同则返回 0 。

算法思想描述：使用广度优先搜索（BFS）来寻找图中最短路径长度。

时间的复杂度： $O(n)$ 。

空间的复杂度： $O(n)$ 。

代码实现：

```
int ShortestPathLength(ALGraph &G, KeyType v, KeyType w) {  
    int start = LocateVex(G, v); // 查找顶点 v 的位置  
    int end = LocateVex(G, w); // 查找顶点 w 的位置  
  
    if (start == -1 || end == -1) {  
        return -1; // 如果顶点 v 或 w 不存在，返回 -1 表示不存在路径  
    }  
  
    if (start == end) {  
        return 0; // 如果起点和终点相同，最短路径长度为 0  
    }  
  
    std::vector<bool> visited(G.vexnum, false); // 记录顶点是否访问过  
    std::vector<int> distance(G.vexnum, 0); // 记录每个顶点到起点的距离  
  
    std::queue<int> q; // 队列用于广度优先搜索  
  
    visited[start] = true;
```

```
q.push(start);
```

```
while (!q.empty()) {
```

```
    int current = q.front();
```

```
    q.pop();
```

```
    ArcNode *p = G.vertices[current].firstarc;
```

```
    while (p != NULL) {
```

```
        int adj = p->adjvex;
```

```
        if (!visited[adj]) {
```

```
            visited[adj] = true;
```

```
            distance[adj] = distance[current] + 1;
```

```
            if (adj == end) {
```

```
                return distance[adj]; // 找到目标顶点，返回最短路
```

径长度

```
            }
```

```
            q.push(adj);
```

```
        }
```

```
        p = p->nextarc;
```

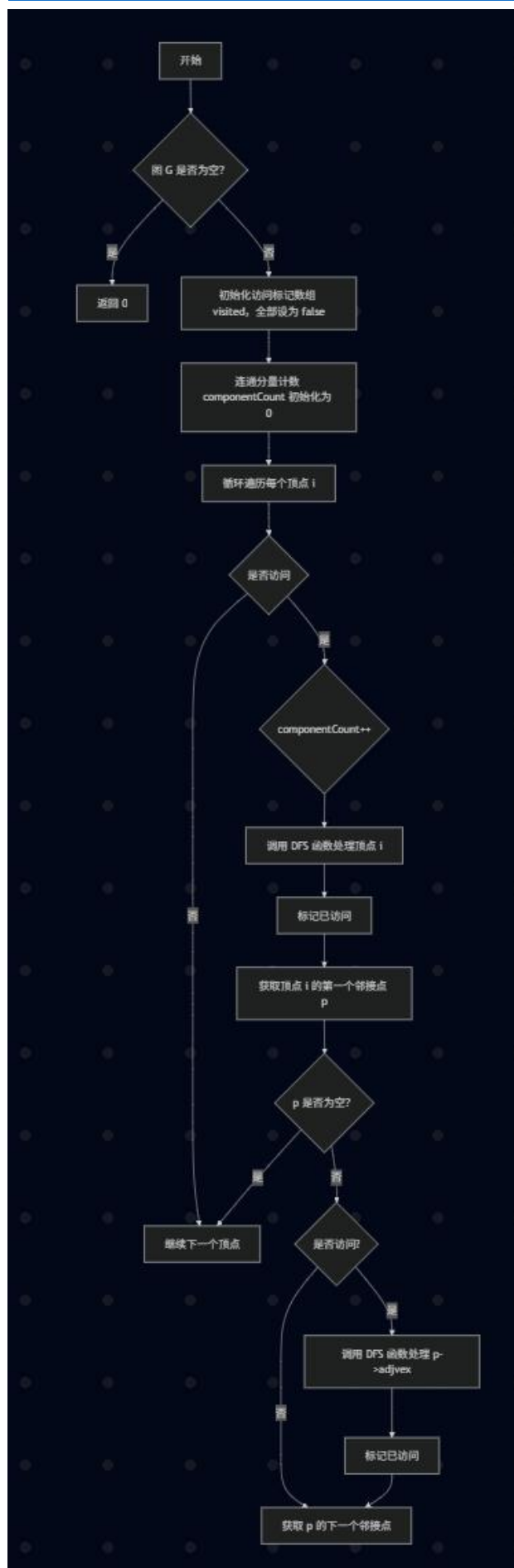
```
    }
```

```
}
```

```
return -1; // 如果无法到达目标顶点，返回 -1 表示不存在路径
```

```
}
```

以下是算法流程图：



2.3 系统实现

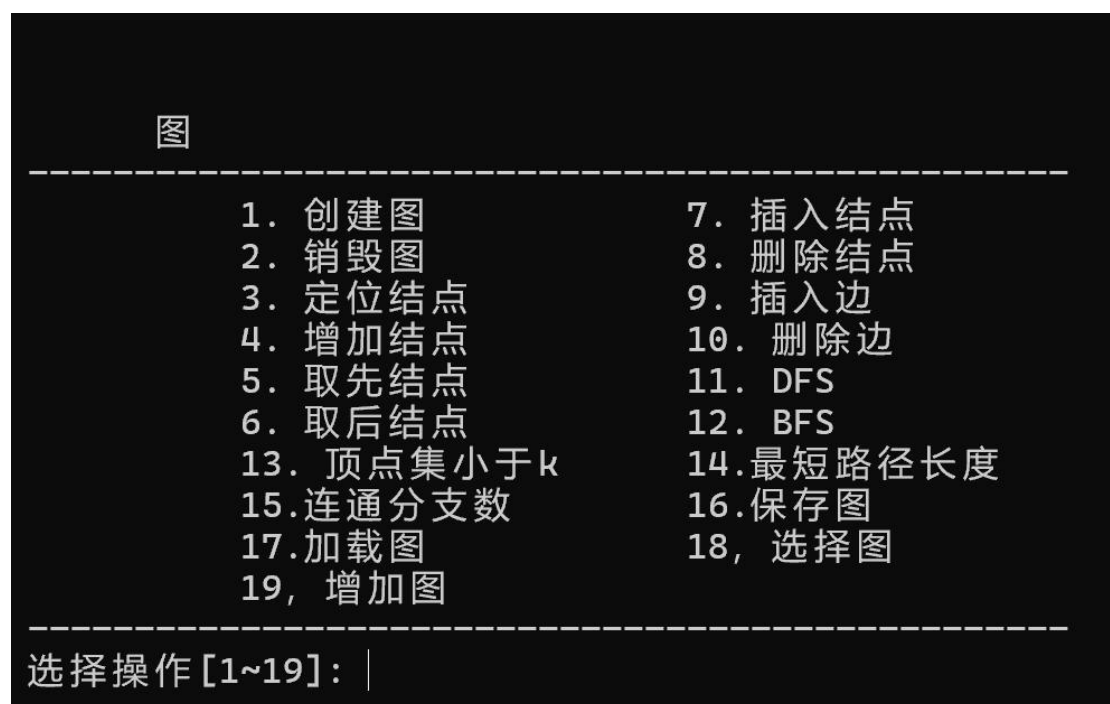
2.3.1 程序开发环境与语言

PC 操作系统为 windows 操作系统使用语言为 C++。

2.3.2 代码的组织结构

演示系统以一个菜单作为交互界面，用户通过输入命令对应的编号来调用相应的函数来实现创建图，销毁图，清空图，插入顶点，删除顶点，遍历等基本操作，以及保存为文件，求最短通路，求距某顶点距离小于 d 的顶点，对顶点进行修改等进阶操作。

程序主函数为一个 switch 结构，根据输入的数字，执行不同的语句，进而调用不同的函数，宏定义函数返回值 ERROR 为 0，INFEASIBLE 为-1，OVERFLOW 为-2，OK 为 1. 交互界面如下图



```
当前没有选择图，请选择操作：
1. 添加图
2. 选择已有图
0. 退出
请输入操作：|
```

2.4 系统测试

程序开发及实现环境：Win11 下使用 Dev C++ 进行编译和调试，开发语言为 C 语言。

表2-1为正常样例测试的输入，预期结果与实际输出

表 2-1 正常样例测试

函数	输入	实际输出	预期结果
创建图	5.5 6 5 7 6 7 7 8 -1 -1	请选择你的操作:1 1. 添加图 2. 选择已有图 3. 退出 4. 销毁图 请输入要添加图的关键字和边:5 6 5 7 6 7 7 8 -1 -1 end	创建无向图成功
定位顶点	8	请选择你的操作:3 请输入要查找顶点的关键字! 8 该顶点的位序是1 其值为 8 集合	返回关键字为 8 的顶点的位序 1
修改顶点	6 9 有向图	请选择你的操作:4 请输入要修改顶点的关键字和修改后的关键字和值! 6 9 有向图 修改成功!	修改成功
查找第一邻接点	7	请选择你的操作:5 请输入要查找第一邻接顶点的关键字! 7 该顶点的第一邻接顶点的位序为1 其值为8 集合	该顶点的第一邻接顶点的位序为 1 其值为 8 集合
查找下一邻接点	7 8	请选择你的操作:6 请输入要查找顶点和相对顶点的关键字! 7 8 集合7的邻接顶点相对于8的下一邻接顶点的位序是3 其值为 6 无向图	顶点 7 的邻接顶点相对于 8 的下一邻接顶点的位序是 3 其值为 6 无向
深度优先遍历	11	请选择你的操作:11 深度优先遍历 5 线性表 7 二叉树 8 集合 6 无向图 end	5 线性表 7 二叉树 8 集合 6 无向图
广度优先遍历	12	请选择你的操作:12 广度优先遍历 5 线性表 7 二叉树 6 无向图 8 集合 end	5 线性表 7 二叉树 6 无向图 8 集合
销毁图	2	请选择你的操作:2 销毁成功!	销毁成功

表2-2为异常样例的输入，实际输出和对输出结果的分析

表 2-2 异常样例测试

异常样例	输入	实际输出	结果分析
创建图	5（略）5 6 5 7 6 7 7 8 -1 -1	请选择你的操作:3 请输入要查找顶点的关键词:5 图不存在,请重试! 无图,无图销毁: 5, 6, 5, 6, 7, 7, 8 -1 -1 图不存在,请重试!	创建无向图成功
定位顶点	10	请选择你的操作:3 请输入要查找顶点的关键词! 10 图中不存在该顶点!	找不到关键字为 10 的顶点
修改顶点	6 5 有向图	请选择你的操作:4 请输入要修改顶点的关键词和修改后的关键词值! 6, 5 有向图 修改失败!	关键字为 5 的结点已存在
查找第一邻接点	10	请选择你的操作:5 请输入要查找第一邻接顶点的关键词! 10 查找失败!	找不到关键字为 10 的顶点
销毁图	2	请选择你的操作:2 销毁成功!	销毁成功
销毁图	2	请选择你的操作:2 销毁失败, 请先创建图!	图还未创建, 无法销毁

通过异常用例可以看出，演示系统对要求插入与已有顶点关键字相同的顶点，和定位不包含在图中的顶点以及销毁还未创建的图等操作的判定能力，可见演示系统能够识别异常样例。

2.5 实验小结

本次实验让我对基于邻接表的图实现的了解更进了一步。演示系统的搭建，让我体会到了主函数和子函数的关系，以及如何搭建一个可以调用不同模块的系统。

在编写插入和删除弧的函数时，如何有效地根据关键字找到相应结点是使程序变得简介的一大关键，在编写插入与删除弧的函数时，如果能够在之前定义定位顶点的函数，并调用，将极大地省去冗杂的代码，这次实验让我对函数的工具性，模块性有了直观的感受。

总的来说，本次数据结构实验提高了我的编程能力，让我对系统整体设计有了更深的认识。

相比之前几次的实验，图的系统实现更为复杂，进阶操作的实现需要了解一些经典的算法，本次实验让我有了较大的进步。

3 课程的收获和建议

3.1 基于链式存储结构的线性表实现

通过对基于链式存储结构的线性表实现的演示系统练习，我基本掌握了线性表的基本操作，能够根据需要调用不同的模块来灵活地使用线性表这一数据结构。

同时在实验过程中，尤其是在 debug 的过程中，我明白了看书看懂了并不代表自己就掌握了一种数据结构，实际上还差的很远，唯有动手实践，多用样例测试，才能了解与熟练运用它。

许许多多的细节问题不通过实践，是无法学到的。

3.2 基于邻接表的图实现

数据结构这门课里最复杂的数据结构就是图，通过对基于邻接表的图实现的实验学习，我对数据结构的认识更进了一步。

这门课程可以说是计算机专业的基础课程，也是核心课程。

经过一学期的学习，我希望未来数据结构课能够越来越好。

参考文献

附录 A 基于链式存储结构线性表实现的源程序

```
//头文件和定义
#include <stdio.h>
#include <malloc.h>
#include <stdlib.h>
#include <windows.h>

#define TRUE 1
#define FALSE 0
#define OK 1
#define ERROR 0
#define INFEASIBLE -1
#define OVERFLOW -2

typedef int status;
typedef int ElemType;

#define LIST_INIT_SIZE 100
#define LISTINCREMENT 10

typedef struct{
    ElemType * elem;
    int length;
    int listsize;
}SqList;

typedef struct{
    struct { char name[30];
            SqList L;
    } elem[10];
    int length;
    int listsize;
}LISTS;

//众多函数的预定义
status InitList(SqList& L);
status DestroyList(SqList& L);
status ClearList(SqList&L);
```

华中科技大学课程实验报告

```
status ListEmpty(SqList L);
int ListLength(SqList L);
status GetElem(SqList L,int i,ElemType& e);
status LocateElem(SqList L,ElemType e);
status PriorElem(SqList L,ElemType cur,ElemType&pre_e);
status NextElem(SqList L,ElemType cur,ElemType&next_e);
status ListInsert(SqList&L,int i,ElemType e);
status ListDelete(SqList&L,int i,ElemType& e);
status ListTraverse(SqList L);
status AddList(LISTS &Lists, char ListName[]);
int LocateList(LISTS Lists,char ListName[]);
status RemoveList(LISTS &Lists,char ListName[]);
status SortList(SqList& L);
status MaxSubArray(SqList& L);
status SubArrayNum(SqList& L,int k);
status SaveList(SqList L, char FileName[]);
status LoadList(SqList &L, char FileName[]);

//主函数
int main(void){
    int op=1,n,e; char s[30]; LISTS Lists;
    Lists.length=0;
    SetConsoleOutputCP(CP_UTF8);
    while(op){
        system("cls");    printf("\n\n");
        printf("                线性表数据结构实验菜单  \n");
        printf("-----\n\n");
        printf("          1.创建线性表          10.插入元素\n");
        printf("          2.销毁线性表          11.删除元素\n");
        printf("          3.清空线性表          12.遍历线性表\n");
        printf("          4.线性表判空          13.最大连续子数组和\n");
        printf("          5.线性表长度          14.和为k的连续子数组\n");
        printf("          6.获取元素            15.线性表排序\n");
        printf("          7.定位元素            16.定位线性表\n");
        printf("          8.前驱元素            17.文件操作\n");
        printf("          9.后驱元素            18.遍历所有线性表\n");
        printf("          0. 退出\n");
        printf("-----\n\n");
        printf("请选择你的操作: [0~18]:\n");
        scanf("%d",&op);
        switch(op){
            case 1:
```

华中科技大学课程实验报告

```
printf("请输入你希望新增的线性表个数: \n");
scanf("%d",&n);
for (int i=0; i<n; i++) {
    printf("请输入第%d个线性表的名称:\n",i+1);
    scanf("%s",s);
    if (AddList(Lists,s)==ERROR) {
        printf("线性表创建失败! \n");
        break;
    }
    printf("请输入这个线性表的元素,元素之间以空格分隔,
以0作结:\n");

    scanf("%d",&e);
    while (e){
        ListInsert(Lists.elem[Lists.length-
1].L,Lists.elem[Lists.length-1].L.length+1,e);
        scanf("%d",&e);
    }
    printf("线性表创建成功! \n");
}
getchar();getchar();
break;
case 2:
    printf("请输入你想要销毁的线性表的名称:\n");
    scanf("%s",s);
    switch (RemoveList(Lists,s)) {
        case INFEASIBLE:
            printf("线性表不存在! \n");
            break;
        case ERROR:
            printf("未找到该线性表! \n");
            break;
        case OK:
            Lists.length--;
            printf("该线性表销毁成功! \n");
            break;
    }
    getchar();getchar();
    break;
case 3:
    printf("请输入你想要清空的线性表的名称: \n");
    scanf("%s",s);
    if (LocateList(Lists,s)==0) printf("未找到该线性
表! \n");
    else {
```

华中科技大学课程实验报告

```
        if(ClearList(Lists.elem[LocateList(Lists,s)
-1].L)==INFEASIBLE) printf("线性表清空失败! \n");
        else printf("线性表清空成功! \n");
    }
    getchar();getchar();
    break;
case 4:
    printf("请输入你希望判空的线性表的名称: \n");
    scanf("%s",s);
    if (LocateList(Lists,s)==0) printf("未找到该线性
表! \n");
    else {
        switch
(ListEmpty(Lists.elem[LocateList(Lists,s)-1].L)) {
            case TRUE:
                printf("线性表为空! \n"); break;
            case FALSE:
                printf("线性表不为空! \n"); break;
            case INFEASIBLE:
                printf("线性表不存在! \n"); break;
        }
    }
    getchar();getchar();
    break;
case 5:
    printf("请输入你希望求长度的线性表的名称: \n");
    scanf("%s",s);
    if (LocateList(Lists,s)==0) printf("未找到该线性
表! \n");
    else {

if(ListLength(Lists.elem[LocateList(Lists,s)-
1].L)==INFEASIBLE) printf("线性表不存在! \n");
        else printf("线性表的长度是
%d!\n",ListLength(Lists.elem[LocateList(Lists,s)-1].L));
    }
    getchar();getchar();
    break;
case 6:
    printf("请输入你想要获取元素的线性表的名称: \n");
    scanf("%s",s);
    if (LocateList(Lists,s)==0) printf("未找到该线性
表! \n");
    else {
```

华中科技大学课程实验报告

```
        int getelem, seq;
        printf("输入你想要获取的线性表元素的序号: \n");
        scanf("%d", &seq);
        switch
(GetElem(Lists.elem[LocateList(Lists, s)-1].L, seq, getelem))
{
            case INFEASIBLE: printf("线性表不存在! \n");
break;

            case ERROR: printf("输入不合法! \n");
break;

            case OK: printf("线性表第%d位的元素是
%d!\n", seq, getelem); break;
        }
    }
    getchar();getchar();
    break;
case 7:
    printf("请输入你希望定位元素的线性表的名称: \n");
    scanf("%s", s);
    if (LocateList(Lists, s)==0) printf("未找到该线性
表! \n");
    else {
        ElemType find_elem;
        printf("请输入你要查找的元素: \n");
        scanf("%d", &find_elem);
        if (LocateElem(Lists.elem[LocateList(Lists, s)
-1].L, find_elem)==INFEASIBLE) printf("线性表不存在!\n");
        if (LocateElem(Lists.elem[LocateList(Lists, s)
-1].L, find_elem)>0) printf("%d元素在线性表的第%d位
!\n", find_elem, LocateElem(Lists.elem[LocateList(Lists, s)-
1].L, find_elem));
        if (LocateElem(Lists.elem[LocateList(Lists, s)
-1].L, find_elem)==0) printf("%d元素在线性表中不存在
!\n", find_elem);
    }
    getchar();getchar();
    break;
case 8:
    printf("请输入你希望获取前驱元素的线性表的名称: \n");
    scanf("%s", s);
    if (LocateList(Lists, s)==0) printf("未找到该线性
表! \n");
    else {
        int f_p_elem, p_elem;
```

华中科技大学课程实验报告

```
        printf("输入你想要查找前驱元素的元素:\n");
        scanf("%d",&f_p_elem);
        switch
(PriorElem(Lists.elem[LocateList(Lists,s)-
1].L,f_p_elem,p_elem)) {
            case INFEASIBLE: printf("线性表不存在! \n");
break;

            case ERROR: printf("该元素没有前驱元素! \n");
break;

            case OK: printf("%d元素的前驱元素是%d!
\n",f_p_elem,p_elem); break;
        }
    }
    getchar();getchar();
    break;
case 9:
    printf("请输入你希望获取后驱元素的线性表的名称: \n");
    scanf("%s",s);
    if (LocateList(Lists,s)==0) printf("未找到该线性
表! \n");
    else {
        int f_n_elem,n_elem;
        printf("输入你想要查找后驱元素的元素:\n");
        scanf("%d",&f_n_elem);
        switch
(NextElem(Lists.elem[LocateList(Lists,s)-
1].L,f_n_elem,n_elem)) {
            case INFEASIBLE: printf("线性表不存在! \n");
break;

            case ERROR: printf("该元素没有后驱元素! \n");
break;

            case OK: printf("%d元素的后驱元素是%d!
\n",f_n_elem,n_elem); break;
        }
    }
    getchar();getchar();
    break;
case 10:
    printf("请输入你希望插入元素的线性表的名称: \n");
    scanf("%s",s);
    if (LocateList(Lists,s)==0) printf("未找到该线性
表! \n");
    else {
        int c_elem,c_seq;
```

```
        printf("请输入你要插入的元素: ");
        scanf("%d",&c_elem);
        printf("你希望把它插到哪个位置: ");
        scanf("%d",&c_seq);
        switch
(ListInsert(Lists.elem[LocateList(Lists,s)-
1].L,c_seq,c_elem)) {
            case INFEASIBLE: printf("线性表不存在! \n");
break;

            case ERROR: printf("插入位置错误! \n");
break;

            case OK: printf("插入成功! \n"); break;
        }
    }
    getchar();getchar();
    break;
case 11:
    printf("请输入你希望删除元素的线性表的名称: \n");
    scanf("%s",s);
    if (LocateList(Lists,s)==0) printf("未找到该线性
表! \n");
    else {
        int d_seq,d_elem;
        printf("请输入你希望删除的元素的序号: ");
        scanf("%d",&d_seq);
        switch
(ListDelete(Lists.elem[LocateList(Lists,s)-
1].L,d_seq,d_elem)) {
            case INFEASIBLE: printf("线性表不存在!\n");
break;

            case ERROR: printf("删除位置不正确! \n");
break;

            case OK: printf("删除成功! \n"); break;
        }
    }
    getchar();getchar();
    break;
case 12:
    printf("请输入你希望遍历的线性表的名称: \n");
    scanf("%s",s);
    if (LocateList(Lists,s)==0) printf("未找到该线性
表! \n");
    else {
```


华中科技大学课程实验报告

```
        if
(!ListTraverse(Lists.elem[LocateList(Lists,s)-1].L))
printf("线性表为空! \n");
    }
    getchar();getchar();
    break;
case 13:
    printf("请输入你希望求最大连续子数组的线性表的名称:
\n");

    scanf("%s",s);
    if (LocateList(Lists,s)==0) printf("未找到该线性
表! \n");

    switch
(MaxSubArray(Lists.elem[LocateList(Lists,s)-1].L)) {
        case INFEASIBLE:
            printf("线性表不存在! \n"); break;
        default:
            printf("该线性表最大连续子数组的和为:
\n%d\n",MaxSubArray(Lists.elem[LocateList(Lists,s)-1].L));
            break;
    }
    getchar(); getchar();
    break;
case 14:
    printf("请输入你希望求连续子数组和为k的线性表的名称:
\n");

    scanf("%s",s);
    if (LocateList(Lists,s)==0) printf("未找到该线性
表! \n");

    else {
        int k;
        printf("请输入数k:\n");
        scanf("%d",&k);
        switch
(SubArrayNum(Lists.elem[LocateList(Lists,s)-1].L,k)) {
            case INFEASIBLE:
                printf("线性表bcz! \n"); break;
            default:
                printf("该线性表连续子数组和为%d的子数
组的数目为:
\n%d\n",k,SubArrayNum(Lists.elem[LocateList(Lists,s)-
1].L,k));

                break;
        }
    }
```

```
        }
        getchar(); getchar();
        break;
    case 15:
        printf("请输入你希望从小到大排序的线性表的名称: \n");
        scanf("%s", s);
        if (LocateList(Lists, s) == 0) printf("未找到该线性
表! \n");
        else {

switch(SortList(Lists.elem[LocateList(Lists, s)-1].L)) {
            case INFEASIBLE:
                printf("线性表不存在! \n"); break;
            case OK:
                printf("排序成功! \n"); break;
        }
    }
    getchar(); getchar();
    break;
    case 16:
        printf("请输入你要定位的线性表的名称: \n");
        scanf("%s", s);
        if (!LocateList(Lists, s)) {
            printf("未找到该线性表! \n");
        } else {
            printf("该线性表的序号为
%d!\n", LocateList(Lists, s));
        }
        getchar(); getchar();
        break;
    case 17:
        printf("请选择操作: \n");
        printf("1. 保存线性表到文件\n");
        printf("2. 从文件加载线性表\n");
        int choice;
        scanf("%d", &choice);
        if (choice == 1) {
            printf("输入你想保存的线性表的名称: \n");
            char listName[30];
            scanf("%s", listName);
            int listIndex = LocateList(Lists,
listName);

            if (listIndex == 0) {
                printf("未找到该线性表! \n");
            }
        }
    }
}
```

```
        } else {
            printf("输入保存的文件名称: \n");
            char s[30];
            scanf("%s", s);
            if (SaveList(Lists.elem[listIndex
- 1].L, s) == OK) {
                printf("线性表已成功保存到文件
%s!\n", s);
            } else {
                printf("保存失败! \n");
            }
        }
    }
    else if (choice == 2) {
        printf("输入加载的文件名称: \n");
        char s[30];
        scanf("%s", s);
        SqList newL;
        if (LoadList(newL, s) == OK) {
            printf("从文件 %s 加载线性表成功! \n", s);
        } else {
            printf("加载失败! \n");
        }
    } else {
        printf("无效的选择! \n");
    }
    getchar(); getchar();
    break;
case 18:
    if (Lists.length<1) printf("无线性表! \n");
    else printf("以下为所有线性表: \n");
    for(int i=0; i<Lists.length; i++) {
        printf("%s:",Lists.elem[i].name);
        ListTraverse(Lists.elem[i].L);
        printf("\n");
    }
    getchar(); getchar();
    break;
case 0:
    Lists.length=0;
    break;
}
}
printf("欢迎下次再使用本系统! \n");
```

```
return 0;
}
```

//以下是众多函数

//函数一

```
status InitList(SqList& L)
// 线性表L不存在, 构造一个空的线性表, 返回OK, 否则返回INFEASIBLE。
{
    if (L.elem!=NULL) return INFEASIBLE;
    else {
        L.elem=(ElemType *)
        malloc(sizeof(ElemType)*LIST_INIT_SIZE);
        L.length=0; L.listsize=LIST_INIT_SIZE;
        return OK;
    }
}
```

//函数二

```
status DestroyList(SqList& L) {
    if (L.elem == NULL) {
        return INFEASIBLE;
    } else {
        free(L.elem);
        L.elem = NULL;
        L.length = 0;
        L.listsize = 0;
        return OK;
    }
}
```

//函数三

```
status ClearList(SqList& L)
// 如果线性表L存在, 删除线性表L中的所有元素, 返回OK, 否则返回
INFEASIBLE。
{
    if (L.elem==NULL) return INFEASIBLE;
    else {
        L.elem=NULL;
        L.elem=(ElemType *)malloc(sizeof(ElemType));
        L.length=0;
        return OK;
    }
}
```

//函数四

status ListEmpty(Sqlist L)

// 如果线性表L存在，判断线性表L是否为空，空就返回TRUE，否则返回FALSE；如果线性表L不存在，返回INFEASIBLE。

```
{
    if (L.elem==NULL) return INFEASIBLE;
    else {
        if (L.length==0) return OK;
        else return ERROR;
    }
}
```

//函数五

status ListLength(Sqlist L)

// 如果线性表L存在，返回线性表L的长度，否则返回INFEASIBLE。

```
{
    if (L.elem==NULL) return INFEASIBLE;
    else return L.length;
}
```

//函数六

status GetElem(Sqlist L,int i,ElemType &e)

// 如果线性表L存在，获取线性表L的第i个元素，保存在e中，返回OK；如果i不合法，返回ERROR；如果线性表L不存在，返回INFEASIBLE。

```
{
    if (L.elem==NULL) return INFEASIBLE;
    else {
        if ((i<1)|| (i>L.length)) return ERROR;
        else {
            e=L.elem[i-1];
            return OK;
        }
    }
}
```

//函数七

int LocateElem(Sqlist L,ElemType e)

// 如果线性表L存在，查找元素e在线性表L中的位置序号并返回该序号；如果e不存在，返回0；当线性表L不存在时，返回INFEASIBLE（即-1）。

```
{
    if (L.elem==NULL) return INFEASIBLE;
    else {
        for (int i=0;i<L.length;i++) {
```

```
        if (L.elem[i]==e) return i+1;
    }
    return 0;
}

//函数八
status PriorElem(SqList L,ElemType e,ElemType &pre)
// 如果线性表L存在, 获取线性表L中元素e的前驱, 保存在pre中, 返回OK;
// 如果没有前驱, 返回ERROR; 如果线性表L不存在, 返回INFEASIBLE。
{
    if (L.elem==NULL) return INFEASIBLE;
    else {
        for (int i=0;i<L.length-1;i++) {
            if (L.elem[i+1]==e) {
                pre=L.elem[i];
                return OK;
            }
        }
        return ERROR;
    }
}

//函数九
status NextElem(SqList L,ElemType e,ElemType &next)
// 如果线性表L存在, 获取线性表L元素e的后继, 保存在next中, 返回OK;
// 如果没有后继, 返回ERROR; 如果线性表L不存在, 返回INFEASIBLE。
{
    if (L.elem==NULL) return INFEASIBLE;
    else {
        for (int i=0;i<L.length-1;i++) {
            if (L.elem[i]==e) {
                next=L.elem[i+1];
                return OK;
            }
        }
        return ERROR;
    }
}

//函数十
status ListInsert(SqList &L,int i,ElemType e)
// 如果线性表L存在, 将元素e插入到线性表L的第i个元素之前, 返回OK; 当
// 插入位置不正确时, 返回ERROR; 如果线性表L不存在, 返回INFEASIBLE。
```

```
{
    if (L.elem==NULL) return INFEASIBLE;
    else {
        if ((i<1)|| (i>L.length+1)) return ERROR;
        else {
            L.length++;
            ElemType *newbase = (ElemType *)realloc(L.elem,
(L.listsize +1) * sizeof(ElemType));
            L.elem = newbase;
            L.listsize++;
            for (int j=L.length-1;j>=i-1;j--) {
                L.elem[j+1]=L.elem[j];
            }
            L.elem[i-1]=e;
            return OK;
        }
    }
}
```

//函数十一

status ListDelete(SqList &L,int i,ElemType &e)
// 如果线性表L存在，删除线性表L的第i个元素，并保存在e中，返回OK；当
删除位置不正确时，返回ERROR；如果线性表L不存在，返回INFEASIBLE。

```
{
    if (L.elem==NULL) return INFEASIBLE;
    else {
        if ((i<1)|| (i>L.length-1)) return ERROR;
        else {
            e=L.elem[i-1];
            for (int j=i;j<L.length;j++) {
                L.elem[j-1]=L.elem[j];
            }
        }
        L.length--;
        return OK;
    }
}
```

//函数十二

status ListTraverse(SqList L)
// 如果线性表L存在，依次显示线性表中的元素，每个元素间空一格，返回OK
；如果线性表L不存在，返回INFEASIBLE。

```
{
    if (L.elem==NULL) return 0;
```

```
else {
    for(int i=0;i<L.length;i++) {
        printf("%d",L.elem[i]);
        if(i<L.length-1) printf(" ");
    }
    return OK;
}
}

//函数十二
status AddList(LISTS &Lists,char ListName[])
// 只需要在Lists中增加一个名称为ListName的空线性表，线性表数据又后台测试程序插入。
{

    if(Lists.length>=10) return ERROR;
    for(int i=0;i<30;i++)
    {
        Lists.elem[Lists.length].name[i]=ListName[i];
    }
    Lists.elem[Lists.length].L.elem=NULL;
    InitList(Lists.elem[Lists.length].L);
    Lists.length++;
    return OK;
}

//函数十三
status RemoveList(LISTS &Lists,char ListName[])
// Lists中删除一个名称为ListName的线性表
{
    for (int i=0; i<Lists.length; i++) {
        if (strcmp(ListName,Lists.elem[i].name)==0) {
            if(DestroyList(Lists.elem[i].L)==INFEASIBLE)
return INFEASIBLE;
            for (int j=i+1;j<Lists.length;j++) {
                Lists.elem[j-1]=Lists.elem[j];
            }
            Lists.length--;
            return OK;
        }
    }
    return ERROR;
}
```


//函数十四

```
int LocateList(LISTS Lists,char ListName[])
// 在Lists中查找一个名称为ListName的线性表，成功返回逻辑序号，否则
返回0
{
    for (int i=0; i<Lists.length; i++) {
        if (strcmp(ListName,Lists.elem[i].name)==0) {
            return i+1;
        }
    }
    return 0;
}
```

//函数十五

```
status SortList(Sqlist& L) {
    if (L.elem == NULL)
        return INFEASIBLE;
    else {
        for (int i = 0; i < L.length - 1; i++) {
            for (int j = 0; j < L.length - 1 - i; j++) {
                int term;
                if (L.elem[j] > L.elem[j + 1]) {
                    term = L.elem[j];
                    L.elem[j] = L.elem[j + 1];
                    L.elem[j + 1] = term;
                }
            }
        }
        return OK;
    }
}
```

//函数十六

```
status MaxSubArray(Sqlist& L) {
    if (L.elem==NULL) return INFEASIBLE;
    else {
        int current=L.elem[0],global=L.elem[0];
        for (int i=1; i<L.length; i++) {

current=((current+L.elem[i]>L.elem[i])?(current+L.elem[i]):
(L.elem[i]));
            if (current >= global) {
                global=current;
            }
        }
    }
}
```

```
        }
    }
    return global;
}

//函数十七
status SubArrayNum(SqList& L,int k) {
    if (L.elem == NULL || L.length == 0) return INFEASIBLE;
    int count = 0;
    for (int i = 0; i < L.length; i++) {
        int current_sum = 0;
        for (int j = i; j < L.length; ++j) {
            current_sum += L.elem[j];
            if (current_sum == k) {
                count++;
            }
        }
    }
    return count;
}
```

//文件读写

//函数十八

```
status SaveList(SqList L, char FileName[]) {
    if (L.elem == NULL) return INFEASIBLE;
    FILE *fp = fopen(FileName, "w");
    if (fp == NULL) return ERROR;

    fprintf(fp, "%d ", L.length);
    for (int i = 0; i < L.length; i++) {
        fprintf(fp, "%d ", L.elem[i]);
    }

    fclose(fp);
    return OK;
}
```

//函数十九

```
status LoadList(SqList &L, char FileName[]) {
    if (L.elem != NULL) return INFEASIBLE;
    FILE *fp = fopen(FileName, "r");
    if (fp == NULL) return ERROR;
```

```
int length;
if (fscanf(fp, "%d", &length) != 1) {
    fclose(fp);
    return ERROR;
}

L.elem = (ElemType *)malloc(length * sizeof(ElemType));
if (L.elem == NULL) {
    fclose(fp);
    return ERROR;
}

for (int i = 0; i < length; i++) {
    if (fscanf(fp, "%d", &L.elem[i]) != 1) {
        free(L.elem);
        L.elem = NULL;
        fclose(fp);
        return ERROR;
    }
}

L.length = length;
L.listsize = length;

fclose(fp);
return OK;
}
```

附录 B 基于邻接表图实现的源程序

```
#include "stdio.h"
#include "stdlib.h"
#include "string.h"
#define TRUE 1
#define FALSE 0
#define OK 1
#define ERROR 0
#define INFEASIBLE -1
#define OVERFLOW -2

typedef int status;
typedef int ElemType; //数据元素类型定义

#define LIST_INIT_SIZE 100
#define LISTINCREMENT 10
typedef int ElemType;
typedef struct LNode{ //单链表（链式结构）结点的定义
    ElemType data;
    struct LNode *next;
}LNode,*LinkList;
typedef struct LLNode{ //链表管理表结点定义
    LinkList L;
    struct LLNode *next;
    char name[32];
}LLNode,*LISTS;
//创建线性表
status InitList(LinkList &L)
// 线性表L不存在，构造一个空的线性表，返回OK，否则返回INFEASIBLE。
{
    if (L!=NULL)
        return INFEASIBLE; //已存在
    L=(LinkList)malloc(sizeof(LNode)); //分配空间
    if (L==NULL)
        return OVERFLOW;
    L->next=NULL;
    return OK;
}

//销毁线性表
status DestroyList(LinkList &L)
// 如果线性表L存在，销毁线性表L，释放数据元素的空间，返回OK，否则返回INFEASIBLE。
{
    if (L==NULL)
        return INFEASIBLE; //不存在
```

```
LNode *p=L,*temp;
if (L->next!=NULL)
{
    p=L->next;
    while (p!=NULL)
    {
        temp=p->next;
        free(p);
        p=temp;
    } //释放到只剩尾结点
}
free(L);
L=NULL; //置空
return OK;
}
```

//清空线性表

```
status ClearList(LinkList &L)
```

// 如果线性表L存在，删除线性表L中的所有元素，返回OK，否则返回INFEASIBLE。

```
{
    if (L==NULL)
        return INFEASIBLE; //不存在
    if (L->next==NULL)
        return OK;
    LNode *p=L->next,*temp;
    while (p!=NULL)
    {
        temp=p->next;
        free(p);
        p=temp;
    }
    L->next=NULL; //置空
    return OK;
}
```

//判空线性表

```
status ListEmpty(LinkList L)
```

// 如果线性表L存在，判断线性表L是否为空，空就返回TRUE，否则返回FALSE；如果线性表L不存在，返回INFEASIBLE。

```
{
    if (L==NULL)
        return INFEASIBLE; //表不存在
    if (L->next==NULL)
        return TRUE;
    return FALSE;
}
```

//求线性表长度

```
int ListLength(LinkList L)
```

华中科技大学课程实验报告

// 如果线性表L存在, 返回线性表L的长度, 否则返回INFEASIBLE。

```
{
    if (L==NULL)
        return INFEASIBLE; //不存在
    int i=0;
    LNode *p;
    p=L->next;
    while (p!=NULL)
    {
        p=p->next;
        i++;
    } //遍历直到p为空
    return i;
}
```

//获取指定位置的元素

status GetElem(LinkList L,int i,ElemType &e)

// 如果线性表L存在, 获取线性表L的第i个元素, 保存在e中, 返回OK; 如果i不合法, 返回ERROR; 如果线性表L不存在, 返回INFEASIBLE。

```
{
    if (L==NULL)
        return INFEASIBLE; //不存在
    int len=1;
    LNode *p=L->next;
    while (p!=NULL)
    {
        p=p->next;
        len++;
        if (len==i)
        {
            e=p->data;
            return OK;
        }
    } //遍历直到p为空
    return ERROR;
}
```

//获取指定元素的位置

status LocateElem(LinkList L,ElemType e)

// 如果线性表L存在, 查找元素e在线性表L中的位置序号; 如果e不存在, 返回ERROR; 当线性表L不存在时, 返回INFEASIBLE。

```
{
    if (L==NULL)
        return INFEASIBLE; //不存在
    int len=1;
    LNode *p=L->next;
    while (p!=NULL)
    {
        if (p->data==e)
            return len; //找到e
    }
}
```

华中科技大学课程实验报告

```
        len++;
        p=p->next;
    }
    return ERROR; //e不存在
}

//获取前驱元素
status PriorElem(LinkList L,ElemType e,ElemType &pre)
// 如果线性表L存在, 获取线性表L中元素e的前驱, 保存在pre中, 返回OK;
// 如果没有前驱, 返回ERROR; 如果线性表L不存在, 返回INFEASIBLE。
{
    if (L==NULL)
        return INFEASIBLE; //不存在
    LNode *p=L->next, *q;
    while (p!=NULL)
    {
        q=p->next;
        if (q!=NULL&&q->data==e) //找到e且位置合法
        {
            pre=p->data;
            return OK;
        }
        p=p->next;
    }
    return ERROR; //未找到e
}

//获取后继元素
status NextElem(LinkList L,ElemType e,ElemType &next)
// 如果线性表L存在, 获取线性表L元素e的后继, 保存在next中, 返回OK;
// 如果没有后继, 返回ERROR; 如果线性表L不存在, 返回INFEASIBLE。
{
    if (L==NULL)
        return INFEASIBLE; //不存在
    LNode *p=L->next, *q;
    while (p!=NULL)
    {
        q=p->next;
        if (q!=NULL&&p->data==e) //找到e且位置合法
        {
            next=q->data;
            return OK;
        }
        p=p->next;
    }
    return ERROR; //未找到e
}

//插入元素
status ListInsert(LinkList &L,int i,ElemType e)
```

华中科技大学课程实验报告

// 如果线性表L存在, 将元素e插入到线性表L的第i个元素之前, 返回OK; 当插入位置不正确时, 返回ERROR; 如果线性表L不存在, 返回INFEASIBLE。

```
{
    if (L==NULL)
        return INFEASIBLE; //不存在
    LNode *p=L->next, *q=L, *newbase;
    int len=1;
    while (p!=NULL)
    {
        if (len==i) //找到合法i
        {
            _INS:
            newbase=(LinkedList)malloc(sizeof(LNode)); //申请
            if (newbase==NULL)
                return OVERFLOW;
            newbase->data=e;
            q->next=newbase;
            newbase->next=p;
            return OK;
        }
        p=p->next;
        q=q->next;
        len++;
    }
    if (len==i)
        goto _INS; //插入位置在最后一个
    return ERROR; //i不合法
}
```

//删除元素

status ListDelete(LinkedList &L, int i, ElemType &e)

// 如果线性表L存在, 删除线性表L的第i个元素, 并保存在e中, 返回OK; 当删除位置不正确时, 返回ERROR; 如果线性表L不存在, 返回INFEASIBLE。

```
{
    if (L==NULL)
        return INFEASIBLE; //不存在
    LNode *p=L->next, *prior=L;
    int len=1;
    while (p!=NULL)
    {
        if (len==i) //找到合法i
        {
            prior->next=p->next;
            e=p->data;
            free(p);
            return OK;
        }
        p=p->next;
        prior=prior->next;
    }
}
```



```
        len++;
    }
    return ERROR; //未找到合法i
}

//遍历线性表
status ListTraverse(LinkList L)
// 如果线性表L存在, 依次显示线性表中的元素, 每个元素间空一格, 返回OK
// 如果线性表L不存在, 返回INFEASIBLE。
{
    if (L==NULL)
        return INFEASIBLE; //不存在
    LNode *p=L->next;
    while (p!=NULL)
    {
        printf("%d ",p->data);
        p=p->next;
    } //遍历输出直到p为空
    return OK;
}

//线性表写文件
status SaveList(LinkList L,char FileName[])
// 如果线性表L存在, 将线性表L的元素写到FileName文件中, 返回OK, 否则返回INFEASIBLE。
{
    if (L==NULL)
        return INFEASIBLE; //不存在
    FILE *fp;
    LNode *p=L->next;
    fp=fopen(FileName,"w");
    while (p!=NULL)
    {
        fprintf(fp,"%d ",p->data);
        p=p->next;
    }
    fprintf(fp,"\n");
    fclose(fp);
    return OK;
}

//线性表读文件
status LoadList(LinkList &L,char FileName[])
// 如果线性表L不存在, 将FileName文件中的数据读入到线性表L中, 返回OK, 否则返回INFEASIBLE。
{
    if (L!=NULL)
        return INFEASIBLE; //已存在表 不能覆写
    L=(LinkList)malloc(sizeof(LNode));
    FILE *fp;
```

```
LNode *p,*prior;
ElemType e;
fp=fopen(FileName,"r");
for(prior=L;fscanf(fp,"%d",&e)==1;prior=prior->next)
{
    p=(LinkedList)malloc(sizeof(LNode));
    p->data=e;
    prior->next=p;
    p->next=NULL;
} //顺序读取直到EOF
fclose(fp);
return OK;
}

//反转线性表
status ReverseList(LinkedList &L)
{
    if(L==NULL)
        return INFEASIBLE; //不存在表
    LNode *p=L->next,*q;
    L->next=NULL;
    while(p!=NULL)
    {
        q=p->next;
        p->next=L->next;
        L->next=p;
        p=q;
    } //原地断裂后首插
    return OK;
}

//删除倒数第i个元素
status RemoveNthFromEnd(LinkedList &L,int i,ElemType &e)
{
    if(L==NULL)
        return INFEASIBLE; //不存在表
    int l=0,n;
    LNode *p=L->next,*q;
    for(;p!=NULL;p=p->next,l++);
    if(i>l||i<1)
        return ERROR; //指定位置非法
    n=l-i+1;
    p=L->next;
    q=L;
    for(int j=1;j<n;j++,p=p->next,q=q->next);
    q->next=p->next;
    e=p->data;
    free(p);
    return OK;
}
```

//将线性表从小到大排序

```
status SortList(LinkList &L)
{
    if (L==NULL)
        return INFEASIBLE; //不存在表
    LNode *p=L->next, *q=L, *pp;
    int len;
    for (len=0; p!=NULL; p=p->next, len++);
    if (len<=1)
        return OK; //只有一个元素
    for (int i=0; i<len-1; i++)
    {
        pp=L;
        p=L->next;
        q=p->next;
        for (int j=0; j<len-i-1; j++, p=p->next, q=q->next, pp=pp->next)
            if (p->data>q->data)
            {
                pp->next=q;
                p->next=q->next;
                q->next=p;
                p=q;
                q=q->next;
            }
    }
    return OK;
}
```

//初始化线性表管理表

```
status InitLists(LISTS &LL)
{
    if (LL!=NULL)
        return INFEASIBLE; //已存在表
    LL=(LLNode*)malloc(sizeof(LLNode));
    if (LL==NULL)
        return OVERFLOW;
    LL->next=NULL;
    return OK;
}
```

//创建名称为name的线性表

```
status CreateList(LISTS &LL, char name[])
{
    if (LL==NULL)
        return INFEASIBLE; //不存在表
    LLNode *P=LL->next, *Q=LL;
    for (; P!=NULL; P=P->next, Q=Q->next)
        if (!strcmp(P->name, name))
            return ERROR; //名称已存在
```

华中科技大学课程实验报告

```
P=(LLNode*)malloc(sizeof(LLNode));
if(P==NULL)
    return OVERFLOW;
Q->next=P;
P->next=NULL;
strcpy(P->name,name);
if(InitList(P->L)==OVERFLOW)
    return OVERFLOW;
return OK;
}

//将当前线性表载入管理表
status WriteList(LISTS &LL,LinkList L,char name[])
{
    if(LL==NULL||L==NULL)
        return INFEASIBLE; //不存在表
    LLNode *P=LL->next,*Q=LL;
    for(;P!=NULL;P=P->next,Q=Q->next)
        if(!strcmp(P->name,name))
            return ERROR; //名称已存在
    P=(LLNode*)malloc(sizeof(LLNode));
    if(P==NULL)
        return OVERFLOW;
    Q->next=P;
    P->next=NULL;
    P->L=L;
    strcpy(P->name,name);
    return OK;
}

//将管理表中的线性表载入当前线性表
status ReadList(LISTS LL,LinkList &L,char name[])
{
    if(LL==NULL)
        return INFEASIBLE; //不存在
    LLNode *P=LL->next;
    int flag;
    for(flag=0;P!=NULL;P=P->next)
        if(P->L==L)
        {
            flag=1;
            break;
        }
    if(!flag)
        return INFEASIBLE; //当前线性表未保存 会造成丢失
    for(P=LL->next;P!=NULL;P=P->next)
        if(!strcmp(name,P->name))
        {
            L=P->L;
            return OK; //找到该表
        }
}
```

```
    return ERROR; //没找到该表
}

//列出当前管理表下的所有线性表
status ListLists(LISTS LL)
{
    if(LL==NULL)
        return INFEASIBLE; //不存在
    LLNode *P=LL->next;
    printf("当前线性表如下: \n");
    for(int i=1;P!=NULL;P=P->next,i++)
        printf("%d %s\n",i,P->name);
    return OK;
}

//定位指定线性表的位置
int LocateList(LISTS LL,char name[])
{
    if(LL==NULL)
        return INFEASIBLE; //不存在
    int i,flag=0;
    LLNode *P=LL->next;
    for(i=1;P!=NULL;P=P->next,i++)
        if(!strcmp(P->name,name))
        {
            flag=i;
            break;
        }
    return flag;
}

//删除指定线性表
status RemoveList(LISTS &LL,LinkList L,char name[])
{
    if(LL==NULL)
        return INFEASIBLE; //不存在
    LLNode *P=LL->next,*Q=LL;
    LNode *p,*q;
    for(;P!=NULL;P=P->next,Q=Q->next)
        if(!strcmp(P->name,name))
        {
            if(P->L!=L) //如果当前使用的线性表不是要被移除的
                线性表 则释放空间
            {
                p=P->L->next;
                while(p!=NULL)
                {
                    q=p->next;
                    free(p);
                    p=q;
                }
            }
        }
}
```

```
        }
        free(P->L);
        P->L=NULL;
    }
    Q->next=P->next;
    free(P);
    return OK;
}
return ERROR; //没找到
}

//销毁管理表
status DeleteLists(LISTS &LL, LinkList L)
{
    if (LL==NULL)
        return INFEASIBLE; //不存在
    LLNode *P=LL->next, *Q;
    LNode *p, *q;
    while (P!=NULL)
    {
        if (P->L!=L) //如果当前使用的线性表不是要被移除的线性表
            则释放空间
        {
            p=P->L->next;
            while (p!=NULL)
            {
                q=p->next;
                free(p);
                p=q;
            }
            free(P->L);
        }
        Q=P->next;
        free(P);
        P=Q;
    }
    free(LL);
    LL=NULL;
    return OK;
}

int main()
{
    LinkList L=NULL; //新建线性表
    LISTS LL=NULL; //新建线性表管理表
    LNode *temp;
    int op=1;
    char name[31], FileName[1024];
    status flag;
    int i, len, out, k, num;
    ElemType e, en, ep; //变量定义
```

```
_LinkList:
op=1;
while (op)
{
    system("cls");
    system("clear");
    printf("\n\n");
    printf("\t单线性表操作菜单 \n");
    printf("-----\n");
    printf("\t1. 创建表\t10. 插入表\n");
    printf("\t2. 销毁表\t11. 删除表\n");
    printf("\t3. 清空表\t12. 遍历表\n");
    printf("\t4. 表判空\t13. 保存表\n");
    printf("\t5. 表长度\t14. 加载表\n");
    printf("\t6. 获取元素\t15. 反转表\n");
    printf("\t7. 定位元素\t16. 删除倒数结点\n");
    printf("\t8. 前驱元素\t17. 表排序\n");
    printf("\t9. 后驱元素\t18. 多列表\n");
    printf("\t0. 退出\n");
    printf("-----\n");
    printf("      请选择你的操作[0~18]:");
    scanf("%d",&op);
    switch(op)
    {
        case 1:
            if (InitList(L)==OK)
                printf("线性表创建成功! \n");
            else
                printf("线性表创建失败! 线性表已存在! \n");
            printf("按回车继续! \n");
            getchar();getchar();
            break;
        case 2:
            if (DestroyList(L)==OK)
                printf("线性表销毁成功! \n");
            else
                printf("线性表销毁失败! 线性表不存在! \n");
            printf("按回车继续! \n");
            getchar();getchar();
            break;
        case 3:
            if (ClearList(L)==OK)
                printf("线性表清空成功! \n");
            else
                printf("线性表清空失败! 线性表不存在! \n");
    }
}
```

```
        printf("按回车继续! \n");
        getchar();getchar();
        break;
    case 4:
        flag=ListEmpty(L);
        if(flag)
            printf("线性表为空! \n");
        else if(!flag)
            printf("线性表不为空! \n");
        else
            printf("线性表判空失败! 线性表不存在! \n");
        printf("按回车继续! \n");
        getchar();getchar();
        break;
    case 5:
        len=ListLength(L);
        if(len!=INFEASIBLE)
            printf("线性表长度为%d! \n",len);
        else
            printf("读取线性表长度失败! 线性表不存在! \n");

        printf("按回车继续! \n");
        getchar();getchar();
        break;
    case 6:
        printf("请输入要获取元素的逻辑位置i! \n");
        scanf("%d",&i);
        flag=GetElem(L,i,e);
        if(flag==INFEASIBLE)
            printf("获取元素失败! 线性表不存在! \n");
        else if(flag==ERROR)
            printf("获取元素失败! 输入的i非法! \n");
        else
            printf("第%d个元素是%d! \n",i,e);
        printf("按回车继续! \n");
        getchar();getchar();
        break;
    case 7:
        printf("请输入需要定位的元素e! \n");
        scanf("%d",&e);
        i=LocateElem(L,e);
        if(i==INFEASIBLE)
            printf("定位失败! 线性表不存在! \n");
        else if(i==ERROR)
            printf("定位失败! 该元素不存在! \n");
        else
            printf("该数字在第%d位! \n",i);
        printf("按回车继续! \n");
        getchar();getchar();
```



```
        break;
    case 8:
        printf("请输入要获取前驱的元素e! \n");
        scanf("%d", &e);
        flag=PriorElem(L,e,ep);
        if(flag==INFEASIBLE)
            printf("获取失败! 线性表不存在! \n");
        else if(flag==ERROR)
        {
            if(e==L->next->data)
                printf("这个元素没有前驱! \n");
            else
                printf("获取失败! 未找到该元素! \n");
        }
        else
            printf("前驱是%d! \n", ep);
        printf("按回车继续! \n");
        getchar();getchar();
        break;
    case 9:
        printf("请输入要获取后继的元素e! \n");
        scanf("%d", &e);
        flag=NextElem(L,e,en);
        if(flag==INFEASIBLE)
            printf("获取失败! 线性表不存在! \n");
        else if(flag==ERROR)
        {
            for(temp=L;temp->next!=NULL;temp=temp-
>next);

            if(L->next!=NULL&&temp->data==e)
                printf("这个元素没有后继! \n");
            else
                printf("获取失败! 未找到该元素! \n");
        }
        else
            printf("后继是%d! \n", en);
        printf("按回车继续! \n");
        getchar();getchar();
        break;
    case 10:
        printf("请输入要插入的位置i和插入的元素e（中间用
空格隔开）! \n");
        scanf("%d %d", &i, &e);
        flag=ListInsert(L,i,e);
        if(flag==INFEASIBLE)
            printf("插入失败! 线性表不存在! \n");
        else if(flag==ERROR)
            printf("插入失败! 插入位置非法! \n");
```

华中科技大学课程实验报告

```
        else if(flag==OVERFLOW)
            printf("插入失败！线性表已满且扩容失败！\n");
        else
            printf("插入成功！\n");
            printf("按回车继续！\n");
            getchar();getchar();
            break;
    case 11:
        printf("请输入要删除元素的逻辑位置i！\n");
        scanf("%d",&i);
        flag=ListDelete(L,i,e);
        if(flag==INFEASIBLE)
            printf("删除失败！线性表不存在！\n");
        else if(flag==ERROR)
            printf("删除失败！删除位置非法！\n");
        else
            printf("删除成功！删除的元素是%d！\n",e);
        printf("按回车继续！\n");
        getchar();getchar();
        break;
    case 12:
        if(!ListTraverse(L))
            printf("线性表是空表！\n");
        else
            printf("\n");
        printf("按回车继续！\n");
        getchar();getchar();
        break;
    case 13:
        printf("请输入保存文件的地址：\n");
        scanf("%s",FileName);
        flag=SaveList(L,FileName);
        if(flag==INFEASIBLE)
            printf("保存错误！线性表不存在！\n");
        else
            printf("保存成功！请查看output.dat！\n");
        printf("按回车继续！\n");
        getchar();getchar();
        break;
    case 14:
        printf("请输入读取文件的地址：\n");
        scanf("%s",FileName);
        flag=LoadList(L,FileName);
        if(flag==INFEASIBLE)
            printf("读取错误！线性表已存在！\n");
        else if(flag==OVERFLOW)
            printf("读取错误！扩容失败！\n");
        else if(flag==ERROR)
```

```
        perror("读取错误！打开文件失败");
    else
        printf("读取成功！\n");
    printf("按回车继续！\n");
    getchar();getchar();
    break;
case 15:
    if(ReverseList(L)==INFEASIBLE)
        printf("反转失败！线性表不存在！\n");
    else
        printf("反转成功！\n");
    printf("按回车继续！\n");
    getchar();getchar();
    break;
case 16:
    printf("请输入要删除的倒数第i个元素的位置i！\n");
    scanf("%d",&i);
    flag=RemoveNthFromEnd(L,i,e);
    if(flag==INFEASIBLE)
        printf("删除失败！线性表不存在！\n");
    else if(flag==ERROR)
        printf("删除失败！位置i非法！\n");
    else
        printf("删除成功！被删除的元素是%d！\n",e);
    printf("按回车继续！\n");
    getchar();getchar();
    break;
case 17:
    if(SortList(L)==INFEASIBLE)
        printf("排序失败！线性表不存在！\n");
    else
        printf("排序成功！\n");
    printf("按回车继续！\n");
    getchar();getchar();
    break;
case 18:
    goto _LISTS;
case 0:
    break;
default:
    printf("输入错误！请重新输入！\n");
    printf("按回车继续！\n");
    getchar();getchar();
    break;
    }
}
_LISTS:
while(op)
{
```

华中科技大学课程实验报告

```
system("cls");
system("clear");
printf("\n\n");
printf("\t线性表管理表操作菜单 \n");
printf("-----
\n");
printf("\t1. 初始化\t5. 表列表\n");
printf("\t2. 创建表\t6. 载入表\n");
printf("\t3. 写入表\t7. 删除表\n");
printf("\t4. 读入表\t8. 销毁表\n");
printf("\t0. 退出\n");
printf("-----
\n");
printf("    请选择你的操作[0~8]:");
scanf("%d",&op);
switch(op)
{
    case 1:
        flag=InitLists(LL);
        if(flag==INFEASIBLE)
            printf("初始化失败！已存在管理表不可操作！
\n");
        else if(flag==OVERFLOW)
            printf("初始化失败！申请空间失败！");
        else
            printf("初始化成功！");
        printf("按回车继续！\n");
        getchar();getchar();
        break;
    case 2:
        printf("请输入线性表名称！\n");
        scanf("%s",name);
        flag=CreateList(LL,name);
        if(flag==INFEASIBLE)
            printf("创建失败！管理表不存在！\n");
        else if(flag==OVERFLOW)
            printf("创建失败！扩容失败！\n");
        else if(flag==ERROR)
            printf("创建失败！名称已存在！\n");
        else
            printf("创建成功！\n");
        printf("按回车继续！\n");
        getchar();getchar();
        break;
    case 3:
        printf("请输入线性表的名称！\n");
        scanf("%s",name);
        flag=WriteList(LL,L,name);
        if(flag==INFEASIBLE)
```

```
{
    if (LL==NULL)
        printf("写入失败！管理表不存在！\n");
    else
        printf("写入失败！当前线性表不存在！\n");
}
else if(flag==ERROR)
    printf("写入失败！名称已存在！\n");
else if(flag==OVERFLOW)
    printf("写入失败！扩容失败！\n");
else
    printf("写入成功！\n");
printf("按回车继续！\n");
getchar();getchar();
break;
case 4:
    printf("请输入要载入线性表的名称！\n");
    scanf("%s",name);
    flag=ReadList(LL,L,name);
    if(flag==INFEASIBLE)
    {
        if(LL==NULL)
            printf("载入失败！管理表不存在！\n");
        else
            printf("当前线性表尚未保存！不可覆写！\n");
    }
    else if(flag==ERROR)
        printf("载入失败！该名称线性表不存在！\n");
    else
        printf("载入成功！\n");
    printf("按回车继续！\n");
    getchar();getchar();
    break;
case 5:
    if(ListLists(LL)==INFEASIBLE)
        printf("列举失败！管理表不存在！\n");
    printf("按回车继续！\n");
    getchar();getchar();
    break;
case 6:
    printf("请输入要定位的线性表名称！\n");
    scanf("%s",name);
    i=LocateList(LL,name);
    if(i==INFEASIBLE)
        printf("定位失败！管理表不存在！\n");
    else if(!i)
        printf("定位失败！该名称线性表不存在！\n");
```

```
        else
            printf("这是第%d个线性表! \n",i);
        printf("按回车继续! \n");
        getchar();getchar();
        break;
    case 7:
        printf("请输入要删除的线性表名称! \n");
        scanf("%s",name);
        flag=RemoveList(LL,L,name);
        if(flag==INFEASIBLE)
            printf("删除失败! 管理表不存在! \n");
        else if(flag==ERROR)
            printf("删除失败! 该名称线性表不存在! \n");
        else
            printf("删除成功! \n");
        printf("按回车继续! \n");
        getchar();getchar();
        break;
    case 8:
        if(DeleteLists(LL,L)==INFEASIBLE)
            printf("销毁失败! 管理表不存在! \n");
        else
            printf("销毁成功! \n");
        printf("按回车继续! \n");
        getchar();getchar();
        break;
    case 0:
        goto _LinkList;
        break;
    default:
        printf("输入错误! 请重新输入! \n");
        printf("按回车继续! \n");
        getchar();getchar();
        break;
    }
}

printf("欢迎下次再使用本系统! \n");
LNode *p,*q;
LLNode *P,*Q;
if(LL!=NULL)
{
    P=LL->next;
    while(P!=NULL)
    {
        p=P->L->next;
        while(p!=NULL)
        {
            q=p->next;
            free(p);
            p=q;
        }
    }
}
```

```
        }
        free(P->L);
        P->L=NULL;
        Q=P->next;
        free(P);
        P=Q;
    }
    free(LL);
    LL=NULL;
}
if (L!=NULL)
{
    p=L->next;
    while (p!=NULL)
    {
        q=p->next;
        free(p);
        p=q;
    }
    free(L);
    L=NULL;
}
printf("按回车继续! \n");
getchar();getchar();
return 0;
}
```

附录 C 基于顺序存储结构的线性表实现

```
#include <stdio.h>
#include <malloc.h>
#include <stdlib.h>
#include <windows.h>
#include <cstring>

#define TRUE 1
#define FALSE 0
#define OK 1
#define ERROR 0
#define INFEASIBLE -1
#define OVERFLOW -2

typedef int status;
typedef int KeyType;
typedef struct {
    KeyType key;
    char others[20];
} TElemType; //二叉树结点类型定义

typedef struct BiTNode{ //二叉链表结点的定义
    TElemType data;
    struct BiTNode *lchild,*rchild;
} BiTNode, *BiTree;

typedef struct {
    char name[50]; // 树的名称
    BiTree root; // 树的根节点
} Tree;

typedef struct {
    Tree trees[100]; // 最多支持 100 棵树
    int length; // 当前树的数量
} TreeManager;

int currentTreeIndex = -1; // 当前选择的树索引

int count=0;
int flag=-1;
int good=0;

void visit(BiTNode T);
int ifsamekey(TElemType a[],int t);
int calculateLength(TElemType definition[]);
```


华中科技大学课程实验报告

```
status CreateBiTree(BiTree &T,TElemType definition[]);
status DestroyBiTree(BiTree &T);
status ClearBiTree(BiTree &T);
status BiTreeEmpty(BiTree &T);
int BiTreeDepth(BiTree T);
BiTNode* LocateNode(BiTree T,KeyType e);
status Assign(BiTree &T,KeyType e,TElemType value);
BiTNode* GetSibling(BiTree T,KeyType e);
status InsertNode(BiTree &T,KeyType e,int LR,TElemType
c);
BiTNode* LocateNodeparent(BiTree T,KeyType e);
BiTree findrs(BiTree T);
void Delete(BiTree &T,BiTree parent,BiTree p);
void Replace(BiTree &T,BiTree parent,BiTree p);
void changeplace(BiTree &T,BiTree parent,BiTree p);
status DeleteNode(BiTree &T,KeyType e);
status PreOrderTraverse(BiTree T,void
(*visit)(BiTree));
status InOrderTraverse(BiTree T,void (*visit)(BiTree));
status PostOrderTraverse(BiTree T,void
(*visit)(BiTree));
status LevelOrderTraverse(BiTree T,void
(*visit)(BiTree));
int MaxPathSum(BiTree T);
BiTNode* LowestCommonAncestor(BiTree T, KeyType e1,
KeyType e2);
status InvertTree(BiTree &T);
void SaveBiTreeHelper(BiTree T, FILE *file);
status SaveBiTree(BiTree T, char FileName[]);
void LoadBiTreeHelper(BiTree &T, FILE *file);
status LoadBiTree(BiTree &T, char FileName[]);

status AddTree(TreeManager &manager, const char *name)
{
    if (manager.length >= 100) {
        return ERROR; // 超过最大树数量
    }
    strcpy(manager.trees[manager.length].name, name);
    manager.trees[manager.length].root = NULL; // 初始
化为空树
    manager.length++;
    return OK;
}

// 管理多棵树
void ManageTrees(TreeManager &manager) {
    while (currentTreeIndex == -1) {
        printf("当前没有选择树, 请选择操作: \n");
        printf("1. 添加树\n");
        printf("2. 选择已有树\n");
    }
}
```

```
printf("0. 退出\n");
printf("请输入操作: ");
int initOp;
scanf("%d", &initOp);
if (initOp == 1) {
    printf("请输入要添加的树名称: ");
    char treeName[50];
    scanf("%s", treeName);
    if (AddTree(manager, treeName) == OK) {
        printf("树 '%s' 添加成功! \n", treeName);
    } else {
        printf("树添加失败! \n");
    }
} else if (initOp == 2) {
    if (manager.length == 0) {
        printf("当前没有可选择的树, 请先添加一个!
\n");
    } else {
        printf("当前已有树: \n");
        for (int j = 0; j < manager.length;
j++) {
            printf("%d. %s\n", j + 1,
manager.trees[j].name);
        }
        printf("请输入要选择的树序号: ");
        int index;
        scanf("%d", &index);
        if (index < 1 || index >
manager.length) {
            printf("无效的树序号! \n");
        } else {
            currentTreeIndex = index - 1;
            printf("已选择树 '%s'\n",
manager.trees[currentTreeIndex].name);
        }
    }
} else if (initOp == 0) {
    printf("程序退出! \n");
    exit(0);
} else {
    printf("无效操作, 请重新输入! \n");
}
}

int main() {
    SetConsoleOutputCP(CP_UTF8);
    TreeManager manager = {0};
    ManageTrees(manager);
}
```

华中科技大学课程实验报告

```
printf("当前选择的树是: %s\n",
manager.trees[currentTreeIndex].name);

BiTree &T = manager.trees[currentTreeIndex].root;
TElemType definition[100];
int op=100;

while(op){
    system("cls");
    printf("\n\n");
    printf("          二叉树数据结构菜单  \n");
    printf("-----\n");
    printf("          1. 创建二叉树          7. 结点的赋值\n");
    printf("          2. 销毁二叉树          8. 兄弟结点\n");
    printf("          3. 清空二叉树          9. 插入结点\n");
    printf("          4. 验空二叉树          10. 删除结点\n");
    printf("          5. 二叉树深度          11. 前序遍历\n");
    printf("          6. 结点的定位          12. 中序遍历\n");
    printf("          13. 后序遍历           14. 按层遍历\n");
    printf("          15. 最大路径和         16. 最近公共祖先\n");
    printf("\n");
    printf("          17. 翻转二叉树          18. 保存二叉树\n");
    printf("\n");
    printf("          19. 加载二叉树          20. 增加二叉树\n");
    printf("\n");
    printf("          21. 删除二叉树          22. 返回总菜单\n");
    printf("\n");
    printf("-----\n");
    printf("-----\n");
    printf("请选择你的操作: [1~21]:");
    scanf("%d", &op);
    switch(op){

        case 1: {
            count=0;
            flag=-1;
            good=0;
            printf("请输入数据, 以关键字为-1结束\n");
            for (int i = 0; i < 100; i++) {
                definition[i].key = 0;
            }
            TElemType definition[100];

            int ans,i=0;
            do {

scanf("%d%s",&definition[i].key,definition[i].others);
```

```
        } while (definition[i++].key!=-1);

        ans=CreateBiTree(T,definition);
        if (ans==OK)
        {
            printf("初始化完成\n");
        }
        else printf("关键字不唯一\n");

        getchar();  getchar();
        break;

    }
    case 2: {
        if (DestroyBiTree(T) == OK) {
            printf("二叉树销毁成功! \n");
        } else {
            printf("二叉树销毁失败! \n");
        }
        getchar(); getchar();
        break;
    }
    case 3: {
        if (ClearBiTree(T) == OK) {
            printf("二叉树清空成功! \n");
        } else {
            printf("二叉树清空失败! \n");
        }
        getchar(); getchar();
        break;
    }
    case 4: {
        if (BiTreeEmpty(T) == OK) {
            printf("二叉树为空! \n");
        } else {
            printf("二叉树不为空! \n");
        }
        getchar(); getchar();
        break;
    }
    case 5: {
        int depth = BiTreeDepth(T);
        printf("二叉树的深度为: %d\n", depth);
        getchar(); getchar();
        break;
    }
    case 6: {
        KeyType e;
```

```
printf("请输入要查找的关键字: \n");
scanf("%d", &e);
BiTNode* node = LocateNode(T, e);
if (node) {
    printf("找到结点: %d, %s\n", node-
>data.key, node->data.others);
} else {
    printf("未找到关键字为 %d 的结点! \n", e);
}
getchar(); getchar();
break;
}
case 7: {
    KeyType e;
    TElemType value;
    printf("请输入要赋值的关键字: \n");
    scanf("%d", &e);
    printf("请输入新值 (key 和 others): \n");
    scanf("%d %s", &value.key, value.others);
    if (Assign(T, e, value) == OK) {
        printf("赋值成功! \n");
    } else {
        printf("赋值失败! \n");
    }
    getchar(); getchar();
    break;
}
case 8: {
    KeyType e;
    printf("请输入要查找兄弟结点的关键字: \n");
    scanf("%d", &e);
    BiTNode* sibling = GetSibling(T, e);
    if (sibling) {
        printf("兄弟结点为: %d, %s\n", sibling-
>data.key, sibling->data.others);
    } else {
        printf("未找到兄弟结点! \n");
    }
    getchar(); getchar();
    break;
}
case 9: {
    KeyType e;
    int LR;
    TElemType c;
    printf("请输入父结点关键字: \n");
    scanf("%d", &e);
    printf("请输入插入位置 (0 表示左, 1 表示右):
\n");
    scanf("%d", &LR);
```

```
printf("请输入插入结点的值 (key 和 others) :
\n");

scanf("%d %s", &c.key, c.others);
if (InsertNode(T, e, LR, c) == OK) {
    printf("插入结点成功! \n");
} else {
    printf("插入结点失败! \n");
}
getchar(); getchar();
break;
}
case 10: {
    KeyType e;
    printf("请输入要删除的结点关键字: \n");
    scanf("%d", &e);
    if (DeleteNode(T, e) == OK) {
        printf("删除结点成功! \n");
    } else {
        printf("删除结点失败! \n");
    }
    getchar(); getchar();
    break;
}
case 11: {
    printf("先序遍历结果: \n");
    PreOrderTraverse(T, visit);
    printf("\n");
    getchar(); getchar();
    break;
}
case 12: {
    printf("中序遍历结果: \n");
    InOrderTraverse(T, visit);
    printf("\n");
    getchar(); getchar();
    break;
}
case 13: {
    printf("后序遍历结果: \n");
    PostOrderTraverse(T, visit);
    printf("\n");
    getchar(); getchar();
    break;
}
case 14: {
    printf("层序遍历结果: \n");
    LevelOrderTraverse(T, visit);
    printf("\n");
    getchar(); getchar();
    break;
}
```

```
    }

    case 15: {
        int maxSum = MaxPathSum(T);
        printf("二叉树的最大路径和为: %d\n", maxSum);
        getchar(); getchar();
        break;
    }
    case 16: {
        KeyType e1, e2;
        printf("请输入两个节点的关键字: \n");
        scanf("%d %d", &e1, &e2);
        BiTNode* ancestor = LowestCommonAncestor(T,
e1, e2);
        if (ancestor) {
            printf("最近公共祖先的关键字为: %d\n",
ancestor->data.key);
        } else {
            printf("未找到最近公共祖先! \n");
        }
        getchar(); getchar();
        break;
    }
    case 17: {
        if (InvertTree(T) == OK) {
            printf("二叉树翻转成功! \n");
        } else {
            printf("二叉树翻转失败! \n");
        }
        getchar(); getchar();
        break;
    }
    case 18: {
        char fileName[100];
        printf("请输入保存文件的名称: \n");
        scanf("%s", fileName);
        if (SaveBiTree(T, fileName) == OK) {
            printf("二叉树保存成功! \n");
        } else {
            printf("二叉树保存失败! \n");
        }
        getchar(); getchar();
        break;
    }
    case 19: {
        char fileName[100];
        printf("请输入加载文件的名称: \n");
        scanf("%s", fileName);
        if (LoadBiTree(T, fileName) == OK) {
            printf("二叉树加载成功! \n");
        }
    }
}
```

```
        } else {
            printf("二叉树加载失败! \n");
        }
        getchar(); getchar();
        break;
    }

    case 20: {
        printf("请输入要添加的树名称: \n");
        char treeName[50];
        scanf("%s", treeName);
        if (AddTree(manager, treeName) == OK) {
            printf("树 '%s' 添加成功! \n", treeName);
        } else {
            printf("树添加失败! 可能已达到最大树数量限制
。 \n");
        }
        getchar(); getchar();
        break;
    }

    case 21: {
        if (manager.length == 0) {
            printf("当前没有可删除的树! \n");
        } else {
            printf("当前已有树: \n");
            for (int j = 0; j < manager.length;
j++) {
                printf("%d. %s\n", j + 1,
manager.trees[j].name);
            }
            printf("请输入要删除的树序号: \n");
            int index;
            scanf("%d", &index);
            if (index < 1 || index >
manager.length) {
                printf("无效的树序号! \n");
            } else {
                // 删除树
                for (int j = index - 1; j <
manager.length - 1; j++) {
                    manager.trees[j] =
manager.trees[j + 1];
                }
                manager.length--;
                currentIndex = -1; // 重置当前选
择的树

                printf("树已删除! \n");
            }
        }
    }
```



```
        }
        getchar(); getchar();
        break;
    }
    case 22: {
        currentTreeIndex = -1; // 重置当前选择的树索引
        ManageTrees(manager); // 调用多树管理函数
        T = manager.trees[currentTreeIndex].root;
        // 更新当前树的根节点
        printf("已返回多树选择界面, 当前选择的树是:
%s\n", manager.trees[currentTreeIndex].name);
        getchar(); getchar();
        break;
    }

    default: {
        printf("无效的操作, 请重新输入! \n");
        getchar(); getchar();
        break;
    }
}

}
```

```
void visit(BiTree T)
{
    printf(" %d,%s",T->data.key,T->data.others);
}

int  ifsamekey(TElemType a[],int t){
    for(int i=0;i<t;i++){
        for(int j=i+1;j<t;j++){
            if(a[i].key==a[j].key&& a[i].key!=0)
                return 1;
        }
    }
    return 0;
}
```

华中科技大学课程实验报告

```
int calculateLength(TElemType definition[]) {
    int length = 0;
    while (definition[length].key != -1) {    // 遇到 key ==
-1 时结束
        length++;
    }
    return length;
}

status CreateBiTree(BiTree &T, TElemType definition[])
/*根据带空枝的二叉树先根遍历序列definition构造一棵二叉树，将根节点
指针赋值给T并返回OK，
如果有相同的关键字，返回ERROR。此题允许通过增加其它函数辅助实现本关任
务*/
{
    // 请在这里补充代码，完成本关任务
    /***** Begin *****/
    TElemType *p=definition;
    int t=calculateLength(definition);
    if(ifsamekey(definition,t)) return ERROR;
    if(definition[count].key==0){
        T=NULL;
        count++;
        return OK;
    }
    if(!(T=(BiTree)malloc(sizeof(BiTNode)))) return ERROR;
    else{
        T->data=definition[count++];
        CreateBiTree(T->lchild,definition);
        CreateBiTree(T->rchild,definition);
        return OK;
    }

    /***** End *****/
}

status ClearBiTree(BiTree &T)
{
    if (T == NULL)
        return OK;    // 空树直接返回

    DestroyBiTree(T->lchild);
    DestroyBiTree(T->rchild);
    T=NULL;

    return OK;
}

status DestroyBiTree(BiTree &T) {
    if (T == NULL) {
        return OK;
    }
}
```

```
    }
    DestroyBiTree(T->lchild); // 递归销毁左子树
    DestroyBiTree(T->rchild); // 递归销毁右子树
    free(T); // 释放当前节点
    T = NULL; // 将当前节点指针置为空
    return OK;
}
status BiTreeEmpty(BiTree &T){
    if(T==NULL) return OK;

    else return ERROR;
}
int BiTreeDepth(BiTree T)
{
    if (T == NULL) {
        return 0;
    }
    int leftDepth = BiTreeDepth(T->lchild);
    int rightDepth = BiTreeDepth(T->rchild);

    return (leftDepth > rightDepth ? leftDepth :
rightDepth) + 1;
}
BiTNode* LocateNode(BiTree T,KeyType e)
//查找结点
{
    // 请在这里补充代码，完成本关任务
    /******* Begin *****/
    if(T==NULL) return NULL;
    if(T->data.key==e) return T;
    BiTree p;
    p=LocateNode(T->lchild,e);
    if(p!=NULL) return p;
    return LocateNode(T->rchild,e);

    /******* End *****/
}
status Assign(BiTree &T,KeyType e,TElemType value)
//实现结点赋值。此题允许通过增加其它函数辅助实现本关任务
{
    // 请在这里补充代码，完成本关任务
    /******* Begin *****/
    BiTree p=LocateNode(T,e);
    BiTree key=NULL;
    if(e!=value.key){
        key=LocateNode(T,value.key);
    }
    if(p==NULL||key!=NULL) return ERROR;
    p->data=value;
}
```

```
return OK;

    /***** End *****/
}
BiTNode* GetSibling(BiTree T,KeyType e)
//实现获得兄弟结点
{
    if (T == NULL) return NULL;
    if (T->data.key == e) return NULL;
    BiTNode* parent = LocateNodeparent(T, e);
    if (parent == NULL) return NULL;
    if (parent->lchild != NULL && parent->lchild->data.key
== e) {
        return parent->rchild;
    } else if (parent->rchild != NULL && parent->rchild-
>data.key == e) {
        return parent->lchild;
    }
    return NULL;
}
status InsertNode(BiTree &T,KeyType e,int LR,TElemType c)
//插入结点。此题允许通过增加其它函数辅助实现本关任务
{
    // 请在这里补充代码，完成本关任务
    /***** Begin *****/
    BiTree p=(BiTree)malloc(sizeof(BiTNode));
    p->data=c;
    p->lchild=NULL;
    p->rchild=NULL;
    if(T==NULL) return ERROR;
    if(LR== -1) {
        p->rchild=T;
        T=p;
        return OK;
    }else{
        BiTree location=NULL;
        location=LocateNode(T,c.key);
        if(location!=NULL) return ERROR;
        location=LocateNode(T,e);
        if(location==NULL) return ERROR;
        if(LR==0){
            p->rchild=location->lchild;
            location->lchild=p;
        }else if(LR==1){
            p->rchild=location->rchild;
            location->rchild=p;
        }
        return OK;
    }
}
```

华中科技大学课程实验报告

```

    /***** End *****/
}
BiTNode* LocateNodeparent(BiTree T,KeyType e)
//查找结点
{
    if (T == NULL) return NULL;
    if (T->lchild != NULL && T->lchild->data.key == e) {
        flag=0;
        return T;
    }
    if (T->rchild != NULL && T->rchild->data.key == e) {
        flag=1;
        return T;}
    BiTNode* leftResult = LocateNodeparent(T->lchild, e);
    if (leftResult != NULL) return leftResult;
    return LocateNodeparent(T->rchild, e);
}
BiTree findrs(BiTree T){
    if(T->rchild==NULL) return T;
    return findrs(T->rchild);
}
void Delete(BiTree &T,BiTree parent,BiTree p){
    if(flag==0){
        parent->lchild=NULL;
        free(p);
        good=1;
    }else if(flag==1){
        parent->rchild=NULL;
        free(p);
        good=1;
    }
}
void Replace(BiTree &T,BiTree parent,BiTree p){
    BiTree newpoint=(p->lchild==NULL?p->rchild:p->lchild);
    if(flag==0){
        parent->lchild=newpoint;
        good=1;
    }else if(flag==1){
        parent->rchild=newpoint;
        good=1;
    }
    free(p);
}
void changeplace(BiTree &T,BiTree parent,BiTree p){
    BiTree rs=findrs(p->lchild);
    if (parent == NULL) {
        rs->rchild = p->rchild;
        T = p->lchild;
        free(p);
        good = 1;
        return;
    }
}
```

```
    }
    if(flag==0){
        parent->lchild=p->lchild;
        rs->rchild=p->rchild;
        good=1;
    }else if(flag==1){
        parent->rchild=p->lchild;
        rs->rchild=p->rchild;
        good=1;
    }
    free(p);
}

status DeleteNode(BiTree &T,KeyType e)
//删除结点。此题允许通过增加其它函数辅助实现本关任务
{
    // 请在这里补充代码，完成本关任务
    /***** Begin *****/
    if(T==NULL) return ERROR;
    BiTree p=LocateNode(T,e);
    if(p==NULL) return ERROR;
    int degree=0;
    if(p->lchild!=NULL) degree++;
    if(p->rchild!=NULL) degree++;
    BiTree parent=LocateNodeparent(T,e);
    switch(degree){
        case 0:
            Delete(T,parent,p);
            break;
        case 1:
            Replace(T,parent,p);
            break;
        case 2:
            changeplace(T,parent,p);
            break;
        default:
            return ERROR;
    }
    if(good) return OK;
    else return ERROR;
    /***** End *****/
}

status PreOrderTraverse(BiTree T,void (*visit)(BiTree))
//先序遍历二叉树T
{
    // 请在这里补充代码，完成本关任务
    /***** Begin *****/
    if(T){
        visit(T);
        PreOrderTraverse(T->lchild,visit);
        PreOrderTraverse(T->rchild,visit);
    }
}
```

华中科技大学课程实验报告

```
    return OK;
    /***** End *****/
}
status InOrderTraverse(BiTree T,void (*visit)(BiTree))
//中序遍历二叉树T
{
    // 请在这里补充代码, 完成本关任务
    /***** Begin *****/
    if(T){
        InOrderTraverse(T->lchild,visit);
        visit(T);
        InOrderTraverse(T->rchild,visit);
    }
    return OK;

    /***** End *****/
}
status PostOrderTraverse(BiTree T,void (*visit)(BiTree))
//后序遍历二叉树T
{
    // 请在这里补充代码, 完成本关任务
    /***** Begin *****/
    if(T){
        PostOrderTraverse(T->lchild,visit);
        PostOrderTraverse(T->rchild,visit);
        visit(T);
    }
    return OK;

    /***** End *****/
}
status LevelOrderTraverse(BiTree T,void (*visit)(BiTree))
//按层遍历二叉树T
{
    // 请在这里补充代码, 完成本关任务
    /***** Begin *****/
    BiTree Queue[10000];
    int rear=0,front=0;
    if(T!=NULL) Queue[rear++]=T;
    while(rear!=front){
        T=Queue[front++];
        visit(T);
        if(T->lchild!=NULL) Queue[rear++]=T->lchild;
        if(T->rchild!=NULL) Queue[rear++]=T->rchild;
    }
    return OK;
    /***** End *****/
}

int MaxPathSum(BiTree T) {
```

```
    if (T == NULL) {
        return 0; // 空节点的路径和为 0
    }
    int leftSum = MaxPathSum(T->lchild); // 递归计算左子树的
    最大路径和
    int rightSum = MaxPathSum(T->rchild); // 递归计算右子树的
    最大路径和
    return T->data.key + (leftSum > rightSum ? leftSum :
    rightSum); // 当前节点值加上左右子树中较大的路径和
}

BiTNode* LowestCommonAncestor(BiTree T, KeyType e1,
KeyType e2) {
    if (T == NULL) {
        return NULL; // 如果当前节点为空, 返回 NULL
    }
    if (T->data.key == e1 || T->data.key == e2) {
        return T; // 如果当前节点是 e1 或 e2, 返回当前节点
    }

    // 递归查找左子树和右子树
    BiTNode* left = LowestCommonAncestor(T->lchild, e1,
e2);
    BiTNode* right = LowestCommonAncestor(T->rchild, e1,
e2);

    if (left != NULL && right != NULL) {
        return T; // 如果 e1 和 e2 分别在左右子树中, 当前节点是
    最近公共祖先
    }

    return (left != NULL) ? left : right; // 否则返回非空的子
    树
}

status InvertTree(BiTree &T) {
    if (T == NULL) {
        return OK; // 空树直接返回
    }

    // 交换当前节点的左右子树
    BiTree temp = T->lchild;
    T->lchild = T->rchild;
    T->rchild = temp;

    // 递归翻转左右子树
    InvertTree(T->lchild);
    InvertTree(T->rchild);
}
```



```
        return OK;
    }

void SaveBiTreeHelper(BiTree T, FILE *file) {
    if (T == NULL) {
        fprintf(file, "0 null\n");
        return;
    }
    fprintf(file, "%d %s\n", T->data.key, T->data.others);
    SaveBiTreeHelper(T->lchild, file);
    SaveBiTreeHelper(T->rchild, file);
}

status SaveBiTree(BiTree T, char FileName[]) {
    FILE *file = fopen(FileName, "w");
    if (file == NULL) {
        return ERROR;
    }
    SaveBiTreeHelper(T, file);
    fclose(file);
    return OK;
}

void LoadBiTreeHelper(BiTree &T, FILE *file) {
    int key;
    char others[20];
    if (fscanf(file, "%d %s", &key, others) != 2) {
        T = NULL;
        return;
    }
    if (key == 0) {
        T = NULL;
        return;
    }
    T = (BiTree)malloc(sizeof(BiTNode));
    T->data.key = key;
    strcpy(T->data.others, others);
    LoadBiTreeHelper(T->lchild, file);
    LoadBiTreeHelper(T->rchild, file);
}

status LoadBiTree(BiTree &T, char FileName[]) {
    FILE *file = fopen(FileName, "r");
    if (file == NULL) {
        return ERROR;
    }
    LoadBiTreeHelper(T, file);
    fclose(file);
    return OK;
}
```

附录 D 基于链表的二叉树实现

```
#include<stdio.h>
#include<cstring>
#include<windows.h>
#include<malloc.h>
#include<stdlib.h>
#include <vector>
#include <queue>
#define TRUE 1
#define FALSE 0
#define OK 1
#define ERROR 0
#define INFEASIBLE -1
#define OVERFLOW -2
#define MAX_VERTEX_NUM 20
typedef int status;
typedef int KeyType;
typedef enum {DG,DN,UDG,UDN} GraphKind;
typedef struct {
    KeyType key;
    char others[20];
} VertexType; //顶点类型定义

typedef struct ArcNode {           //表结点类型定义
    int adjvex;                   //顶点位置编号
    struct ArcNode *nextarc;      //下一个表结点指针
} ArcNode;

typedef struct VNode{             //头结点及其数组类型定义
    VertexType data;              //顶点信息
    ArcNode *firstarc;            //指向第一条弧
} VNode,AdjList[MAX_VERTEX_NUM];
typedef struct { //邻接表的类型定义
    AdjList vertices;             //头结点数组
    int vexnum,arcnum;            //顶点数、弧数
    GraphKind kind;               //图的类型
} ALGraph;

void visit(VertexType v)
{
    printf(" %d %s",v.key,v.others);
}

int Numof_arcnum(KeyType a[][2]);
int Numof_vexnum(VertexType a[]);
int isunique(VertexType a[],int vexnum);
```

华中科技大学课程实验报告

```
status CreateCraph(ALGraph &G,VertexType
V[],KeyType VR[][2]);
status DestroyGraph(ALGraph &G);
int LocateVex(ALGraph G,KeyType u);
int isuniqueofG(ALGraph G);
status PutVex(ALGraph &G,KeyType u,VertexType
value);
int FirstAdjVex(ALGraph G,KeyType u);
int NextAdjVex(ALGraph G, KeyType v, KeyType w);
status InsertVex(ALGraph &G,VertexType v);
status DeleteVex(ALGraph &G, KeyType v);
int isuniqueofG(ALGraph G,int i,int j);
status InsertArc(ALGraph &G,KeyType v,KeyType w);
status DeleteArc(ALGraph &G,KeyType v,KeyType w);
void DFS(ALGraph G,int i,bool visited[],void
(*visit)(VertexType));
status DFSTraverse(ALGraph &G,void
(*visit)(VertexType));
status BFSTraverse(ALGraph &G,void
(*visit)(VertexType));
std::vector<int> VerticesSetLessThanK(ALGraph &G,
KeyType v, int k) ;
int ShortestPathLength(ALGraph &G, KeyType v,
KeyType w);
int ConnectedComponentsNums(ALGraph &G);
status SaveGraph(ALGraph G, char FileName[]);
status LoadGraph(ALGraph &G, char FileName[]);
void visit(VertexType v);

typedef struct {
    ALGraph graphs[MAX_VERTEX_NUM]; // 存储多个图
    char names[MAX_VERTEX_NUM][50]; // 存储每个图的名称
    int length; // 当前图的数量
} GraphManager;

void ManageGraphs(GraphManager &manager) {
    int currentGraphIndex = -1;

    while (currentGraphIndex == -1) {
        printf("当前没有选择图, 请选择操作: \n");
        printf("1. 添加图\n");
        printf("2. 选择已有图\n");
        printf("0. 退出\n");
        printf("请输入操作: ");
        int initOp;
        scanf("%d", &initOp);

        if (initOp == 1) {
            if (manager.length >= MAX_VERTEX_NUM) {
                printf("图的数量已达到上限, 无法添加更多图! \n");
            }
        }
    }
}
```

```
        continue;
    }

    printf("请输入要添加的图名称: ");
    char graphName[50];
    scanf("%s", graphName);

    // 添加图但不初始化
    manager.graphs[manager.length].vexnum = 0;
    manager.graphs[manager.length].arcnum = 0;
    manager.graphs[manager.length].kind = UDG; //
默认类型
    strcpy(manager.names[manager.length],
graphName); // 保存图名称
    manager.length++;
    printf("图 '%s' 添加成功! \n", graphName);
} else if (initOp == 2) {
    if (manager.length == 0) {
        printf("当前没有可选择的图, 请先添加一个! \n");
    } else {
        printf("当前已有图: \n");
        for (int j = 0; j < manager.length; j++) {
            printf("%d. %s\n", j + 1,
manager.names[j]); // 显示图的名称
        }
        printf("请输入要选择的图序号: ");
        int index;
        scanf("%d", &index);
        if (index < 1 || index > manager.length) {
            printf("无效的图序号! \n");
        } else {
            currentGraphIndex = index - 1;
            printf("已选择图 '%s'\n",
manager.names[currentGraphIndex]);
        }
    }
} else if (initOp == 0) {
    printf("程序退出! \n");
    exit(0);
} else {
    printf("无效操作, 请重新输入! \n");
}
}

printf("当前选择的图是: %s\n",
manager.names[currentGraphIndex]);
}
int SelectGraph(GraphManager &manager) {
    if (manager.length == 0) {
```

华中科技大学课程实验报告

```
printf("当前没有可选择的图，请先添加一个！\n");
return -1; // 返回 -1 表示没有可选择的图
}

printf("当前已有图：\n");
for (int i = 0; i < manager.length; i++) {
    printf("%d. 图 %d\n", i + 1, i + 1);
}

printf("请输入要选择的图序号：");
int index;
scanf("%d", &index);

if (index < 1 || index > manager.length) {
    printf("无效的图序号！\n");
    return -1; // 返回 -1 表示选择无效
}

printf("已选择图 %d\n", index);
return index - 1; // 返回图的索引
}

int main() {
    SetConsoleOutputCP(CP_UTF8);
    GraphManager manager = {0};
    ManageGraphs(manager);
    ALGraph G;
    int op = 100;

    while (op) {
        system("cls");
        printf("\n\n");
        printf("          图数据结构实验菜单\n");
        printf("-----\n");
        printf("          1. 创建图          7. 插入结点\n");
        printf("          2. 销毁图          8. 删除结点\n");
        printf("          3. 定位结点        9. 插入边\n");
        printf("          4. 增加结点        10. 删除边\n");
        printf("          5. 取先结点        11. DFS\n");
        printf("          6. 取后结点        12. BFS\n");
        printf("          13. 顶点集小于k    14. 最短路径长度\n");
        printf("          15. 连通分支数     16. 保存图\n");
        printf("          17. 加载图         18. 选择图\n");
    }
}
```

华中科技大学课程实验报告

```
printf("          19.增加图\n");
printf("-----\n");
printf("选择操作[1~19]: ");
scanf("%d", &op);

switch (op) {
    case 1: { // 创建图
        VertexType V[100];
        KeyType VR[100][2];
        printf("请输入顶点信息 (key 和 others),
以 key=-1 结束: \n");
        int i = 0;
        do {
            scanf("%d%s", &V[i].key,
V[i].others);
        } while (V[i++].key != -1);

        printf("请输入边信息 (起点和终点 key), 以
-1 -1 结束: \n");
        int j = 0;
        do {
            scanf("%d%d", &VR[j][0],
&VR[j][1]);
        } while (VR[j++][0] != -1);

        if (CreateCraph(G, V, VR) == OK) {
            printf("图创建成功! \n");
        } else {
            printf("图创建失败! \n");
        }
        getchar(); getchar();
        break;
    }
    case 2: { // 销毁图
        if (DestroyGraph(G) == OK) {
            printf("图销毁成功! \n");
        } else {
            printf("图销毁失败! \n");
        }
        getchar(); getchar();
        break;
    }
    case 3: { // 查找顶点
        KeyType u;
        printf("请输入要查找的顶点关键字: \n");
        scanf("%d", &u);
        int pos = LocateVex(G, u);
        if (pos != -1) {
```

华中科技大学课程实验报告

```
printf("顶点 '%d' 的位置序号为: %d\n",
u, pos);
    } else {
        printf("未找到关键字为 '%d' 的顶点!
\n", u);
    }
    getchar(); getchar();
    break;
}
case 4: { // 顶点赋值
    KeyType u;
    VertexType value;
    printf("请输入要赋值的顶点关键字: \n");
    scanf("%d", &u);
    printf("请输入新值 (key 和 others): \n");
    scanf("%d%s", &value.key,
value.others);
    if (PutVex(G, u, value) == OK) {
        printf("赋值成功! \n");
    } else {
        printf("赋值失败! \n");
    }
    getchar(); getchar();
    break;
}
case 5: { // 获得第一邻接点
    KeyType u;
    printf("请输入顶点关键字: \n");
    scanf("%d", &u);
    int firstAdj = FirstAdjVex(G, u);
    if (firstAdj != -1) {
        printf("顶点 '%d' 的第一个邻接点位序为
: %d\n", u, firstAdj);
    } else {
        printf("顶点 '%d' 没有邻接点或不存在!
\n", u);
    }
    getchar(); getchar();
    break;
}
case 6: { // 获得下一邻接点
    KeyType v, w;
    printf("请输入顶点 v 和邻接点 w 的关键字:
\n");
    scanf("%d%d", &v, &w);
    int nextAdj = NextAdjVex(G, v, w);
    if (nextAdj != -1) {
        printf("顶点 '%d' 的相对于 '%d' 的下
一个邻接点位序为: %d\n", v, w, nextAdj);
```

```
        } else {
            printf("未找到下一个邻接点或顶点不存在
! \n");

        }
        getchar(); getchar();
        break;
    }
    case 7: { // 插入顶点
        VertexType v;
        printf("请输入要插入的顶点信息 (key 和
others): \n");

        scanf("%d%s", &v.key, v.others);
        if (InsertVex(G, v) == OK) {
            printf("顶点插入成功! \n");
        } else {
            printf("顶点插入失败! \n");
        }
        getchar(); getchar();
        break;
    }
    case 8: { // 删除顶点
        KeyType v;
        printf("请输入要删除的顶点关键字: \n");
        scanf("%d", &v);
        if (DeleteVex(G, v) == OK) {
            printf("顶点删除成功! \n");
        } else {
            printf("顶点删除失败! \n");
        }
        getchar(); getchar();
        break;
    }
    case 9: { // 插入弧
        KeyType v, w;
        printf("请输入要插入的弧 (起点 key 和终点
key): \n");

        scanf("%d%d", &v, &w);
        if (InsertArc(G, v, w) == OK) {
            printf("弧插入成功! \n");
        } else {
            printf("弧插入失败! \n");
        }
        getchar(); getchar();
        break;
    }
    case 10: { // 删除弧
        KeyType v, w;
        printf("请输入要删除的弧 (起点 key 和终点
key): \n");
```



```
scanf("%d%d", &v, &w);
if (DeleteArc(G, v, w) == OK) {
    printf("弧删除成功! \n");
} else {
    printf("弧删除失败! \n");
}
getchar(); getchar();
break;
}
case 11: { // 深度优先遍历
    KeyType startKey;
    printf("请输入深度优先遍历的起始顶点关键字:
\n");

    scanf("%d", &startKey);

    int startIndex = LocateVex(G,
startKey);

    if (startIndex == -1) {
        printf("起始顶点 '%d' 不存在! \n",
startKey);

    } else {
        printf("深度优先遍历结果: \n");
        bool visited[MAX_VERTEX_NUM] = {false};
        DFS(G, startIndex, visited, visit);
        printf("\n遍历完成! \n");
    }
    getchar(); getchar();
    break;
}
case 12: { // 广度优先遍历
    KeyType startKey;
    printf("请输入广度优先遍历的起始顶点关键字:
\n");

    scanf("%d", &startKey);

    int startIndex = LocateVex(G,
startKey);

    if (startIndex == -1) {
        printf("起始顶点 '%d' 不存在! \n",
startKey);

    } else {
        printf("广度优先遍历结果: \n");
        bool visited[MAX_VERTEX_NUM] = {false};
        int QUEUE[MAX_VERTEX_NUM], head = 0,
tail = 0;

        visited[startIndex] = true;
        visit(G.vertices[startIndex].data);
        QUEUE[tail++] = startIndex;
```

华中科技大学课程实验报告

```
        while (head != tail) {
            int current = QUEUE[head++];
            ArcNode *p =
G.vertices[current].firstarc;
            while (p != NULL) {
                if (!visited[p->adjvex]) {
                    visited[p->adjvex] = true;
                    visit(G.vertices[p-
>adjvex].data);
                    QUEUE[tail++] = p->adjvex;
                }
                p = p->nextarc;
            }
        }
        printf("\n遍历完成! \n");
    }

    getchar(); getchar();
    break;
}

case 13: { // 距离小于 k 的顶点集合
    KeyType v;
    int k;
    printf("请输入起始顶点关键字: \n");
    scanf("%d", &v);
    printf("请输入距离阈值 k: \n");
    scanf("%d", &k);
    std::vector<int> result =
VerticesSetLessThanK(G, v, k);
    if (result.empty()) {
        printf("没有与顶点 %d 距离小于 %d 的顶
点! \n", v, k);
    } else {
        printf("与顶点 %d 距离小于 %d 的顶点集
合为: ", v, k);

        for (int key : result) {
            printf("%d ", key);
        }
        printf("\n");
    }
    getchar(); getchar();
    break;
}

case 14: { // 顶点间最短路径长度
    KeyType v, w;
    printf("请输入起始顶点关键字: \n");
    scanf("%d", &v);
    printf("请输入目标顶点关键字: \n");
    scanf("%d", &w);
    int length = ShortestPathLength(G, v,
w);
```

华中科技大学课程实验报告

```
        if (length != -1) {
            printf("顶点 %d 和顶点 %d 之间的最短路
径长度为: %d\n", v, w, length);
        } else {
            printf("顶点 %d 和顶点 %d 之间不存在路
径! \n", v, w);
        }
        getchar(); getchar();
        break;
    }
    case 15: { // 图的连通分量个数
        int numComponents =
ConnectedComponentsNums(G);
        printf("图的连通分量个数为: %d\n",
numComponents);

        getchar(); getchar();
        break;
    }
    case 16: { // 保存图到文件
        char fileName[100];
        printf("请输入保存文件的名称: \n");
        scanf("%s", fileName);
        if (SaveGraph(G, fileName) == OK) {
            printf("图保存成功! \n");
        } else {
            printf("图保存失败! \n");
        }
        getchar(); getchar();
        break;
    }
    case 17: { // 从文件加载图
        char fileName[100];
        printf("请输入加载文件的名称: \n");
        scanf("%s", fileName);
        if (LoadGraph(G, fileName) == OK) {
            printf("图加载成功! \n");
        } else {
            printf("图加载失败! \n");
        }
        getchar(); getchar();
        break;
    }
    case 18: {
        if (manager.length == 0) {
            printf("当前没有可选择的图, 请先添加一
个! \n");
        } else {
            printf("当前已有图: \n");
        }
    }
}
```

华中科技大学课程实验报告

```

                                for (int i = 0; i < manager.length;
i++) {
                                printf("%d. %s\n", i + 1,
manager.names[i]);
                                }

                                printf("请输入要选择的图序号: ");
                                int selectedGraphIndex;
                                scanf("%d", &selectedGraphIndex);

                                if (selectedGraphIndex < 1 ||
selectedGraphIndex > manager.length) {
                                printf("无效的图序号! \n");
                                } else {
                                G = manager.graphs[selectedGraphIndex
- 1];

                                printf("已选择图 '%s'\n",
manager.names[selectedGraphIndex - 1]);
                                }
                                }
                                getchar(); getchar();
                                break;
                                }
                                case 19: { // 添加新图
                                if (manager.length >= MAX_VERTEX_NUM)
{
                                printf("图的数量已达到上限, 无法添加更
多图! \n");

                                } else {
                                printf("请输入要添加的图名称: ");
                                char graphName[50];
                                scanf("%s", graphName);

                                manager.graphs[manager.length].vexnum
= 0;
                                manager.graphs[manager.length].arcnum
= 0;
                                manager.graphs[manager.length].kind =
UDG;
                                strcpy(manager.names[manager.length],
graphName);
                                manager.length++;

                                printf("图 '%s' 添加成功! 当前图总数:
%d\n", graphName, manager.length);
                                }
                                getchar(); getchar();
                                break;
                                }

```

```
        default: {
            printf("无效的操作, 请重新输入! \n");
            getchar(); getchar();
            break;
        }
    }
}

return 0;
}

int Numof_arcnum(KeyType a[][2]){
    int i=0;
    while(a[i][0]!=-1){
        i++;
    }
    return i;
}

int Numof_vexnum(VertexType a[]){
    int i=0;
    while(a[i].key!=-1){
        i++;
    }
    return i;
}

int isunique(VertexType a[],int vexnum){
    int j;
    for(int i=0;i<vexnum;i++){
        for(j=i+1;j<vexnum;j++){
            if(a[i].key==a[j].key) return 0;
        }
    }
    return 1;
}

status CreateCraph(ALGraph &G,VertexType V[],KeyType
VR[][2])
/*根据V和VR构造图T并返回OK, 如果V和VR不正确, 返回ERROR
如果有相同的关键字, 返回ERROR。此题允许通过增加其它函数辅助实现本关任
务*/
{
    // 请在这里补充代码, 完成本关任务
    /***** Begin *****/
    G.arcnum=Numof_arcnum(VR);
    G.vexnum=Numof_vexnum(V);
    if(G.vexnum==0) return ERROR;
    if(!isunique(V,G.vexnum)){
        return ERROR;
    }
    for(int i=0;i<G.vexnum;i++){
        G.vertices[i].data=V[i];
        G.vertices[i].firstarc=NULL;
    }
}
```

```
}
int m,n;
for(int i=0;i<G.arcnum;i++){
    for(m=0;m<G.vexnum;m++){
        if(G.vertices[m].data.key==VR[i][0])
            break;
    }
    for(n=0;n<G.vexnum;n++){
        if(G.vertices[n].data.key==VR[i][1])
            break;
    }
    if(m>=G.vexnum||n>=G.vexnum) return ERROR;
    ArcNode *p=(ArcNode *)malloc(sizeof(ArcNode));
    p->adjvex=n;
    p->nextarc=G.vertices[m].firstarc;
    G.vertices[m].firstarc=p;
    p=(ArcNode *)malloc(sizeof(ArcNode));
    p->adjvex=m;
    p->nextarc=G.vertices[n].firstarc;
    G.vertices[n].firstarc=p;
}
return OK;

/***** End *****/
}
status DestroyGraph(ALGraph &G)
/*销毁无向图G,删除G的全部顶点和边*/
{
    // 请在这里补充代码,完成本关任务
    /***** Begin *****/
    if (G.vexnum == 0 && G.arcnum == 0) {
        return OK;
    }
    for(int i=0;i<G.vexnum;i++){
        ArcNode *p=G.vertices[i].firstarc;
        while(p!=NULL){
            ArcNode *temp=p;
            p=p->nextarc;
            free(temp);
        }
        G.vertices[i].firstarc = NULL;
    }
    G.arcnum=0;
    G.vexnum=0;
    return OK;
    /***** End *****/
}
int LocateVex(ALGraph G,KeyType u)
//根据u在图G中查找顶点,查找成功返回位序,否则返回-1;
{
```

```
// 请在这里补充代码，完成本关任务
/***** Begin *****/
for(int i=0;i<G.vexnum;i++){
    if(G.vertices[i].data.key==u) return i;
}
return -1;
/***** End *****/
}
int isuniqueofG(ALGraph G){
    int i,j;
    for(i=0;i<G.vexnum;i++){
        for(j=i+1;j<G.vexnum;j++){

if(G.vertices[i].data.key==G.vertices[j].data.key)    return
0;

        }
    }
    return 1;
}
status PutVex(ALGraph &G,KeyType u,VertexType value)
//根据u在图G中查找顶点，查找成功将该顶点值修改成value，返回OK;
//如果查找失败或关键字不唯一，返回ERROR
{
    // 请在这里补充代码，完成本关任务
    /***** Begin *****/
    int flag;
    if((flag=LocateVex(G,u))== -1||!isuniqueofG(G)) return
ERROR;
    G.vertices[flag].data=value;
    if(!isuniqueofG(G)) return ERROR;
    return OK;
    /***** End *****/
}
int FirstAdjVex(ALGraph G,KeyType u)
//根据u在图G中查找顶点，查找成功返回顶点u的第一邻接顶点位序，否则返回
-1;
{
    // 请在这里补充代码，完成本关任务
    /***** Begin *****/
    int i=LocateVex(G,u);
    if(i== -1) return ERROR;
    if(G.vertices[i].firstarc==NULL) return ERROR;
    return G.vertices[i].firstarc->adjvex;

    /***** End *****/
}
int NextAdjVex(ALGraph G, KeyType v, KeyType w)
    // v 对应 G 的一个顶点，w 对应 v 的邻接顶点；操作结果是返回
v 的（相对于 w）下一个邻接顶点的位序;
```

华中科技大学课程实验报告

// 如果 w 是最后一个邻接顶点, 或 v、w 对应顶点不存在, 则返回 -1。

```
{
    int i = LocateVex(G, v);
    if (i == -1) return -1;

    ArcNode *p = G.vertices[i].firstarc;
    while (p != NULL) {
        if (p->adjvex == LocateVex(G, w)) {
            if (p->nextarc != NULL) {
                return p->nextarc->adjvex;
            } else {
                return -1;
            }
        }
        p = p->nextarc;
    }

    return -1;
}

status InsertVex(ALGraph &G, VertexType v)
//在图G中插入顶点v, 成功返回OK, 否则返回ERROR
{
    // 请在这里补充代码, 完成本关任务
    /***** Begin *****/
    G.vertices[G.vexnum].data=v;
    G.vertices[G.vexnum].firstarc=NULL;
    G.vexnum++;
    if(!isuniqueofG(G)) return ERROR;
    if(G.vexnum>MAX_VERTEX_NUM) return ERROR;
    return OK;
    /***** End *****/
}

status DeleteVex(ALGraph &G, KeyType v) {
    int i = LocateVex(G, v);
    if (i == -1) return ERROR;
    if (G.vexnum == 1) return ERROR;

    ArcNode *p = G.vertices[i].firstarc;
    while (p != NULL) {
        ArcNode *temp = p;
        p = p->nextarc;
        free(temp);
        G.arcnum--;
    }

    for (int j = 0; j < G.vexnum; j++) {
        if (j == i) continue;

        ArcNode *prev = NULL;
```



```
ArcNode *curr = G.vertices[j].firstarc;

while (curr != NULL) {
    if (curr->adjvex == i) {
        if (prev == NULL) {
            G.vertices[j].firstarc = curr->nextarc;
            free(curr);
            curr = G.vertices[j].firstarc;
        } else {
            prev->nextarc = curr->nextarc;
            free(curr);
            curr = prev->nextarc;
        }
    } else {
        if (curr->adjvex > i) {
            curr->adjvex--;
        }
        prev = curr;
        curr = curr->nextarc;
    }
}

for (int j = i; j < G.vexnum - 1; j++) {
    G.vertices[j] = G.vertices[j + 1];
}

G.vexnum--;
return OK;
}

int isuniquenew(ALGraph G,int i,int j){
    ArcNode *p=G.vertices[i].firstarc;
    while(p!=NULL){
        if(p->adjvex==j)    return 0;
        else{
            p=p->nextarc;
        }
    }
    return 1;
}

status InsertArc(ALGraph &G,KeyType v,KeyType w)
//在图G中增加弧<v,w>, 成功返回OK, 否则返回ERROR
{
    // 请在这里补充代码, 完成本关任务
    /***** Begin *****/
    int m=LocateVex(G,v),n=LocateVex(G,w);
    if(m==-1||n==-1)    return ERROR;
    if(!isuniquenew(G,m,n))    return ERROR;
    ArcNode *p=(ArcNode *)malloc(sizeof(ArcNode));
```

```
p->nextarc=G.vertices[m].firstarc;
G.vertices[m].firstarc=p;
p->adjvex=n;
p=(ArcNode *)malloc(sizeof(ArcNode));
p->nextarc=G.vertices[n].firstarc;
G.vertices[n].firstarc=p;
p->adjvex=m;
G.arcnum++;
return OK;
/***** End *****/
}
status DeleteArc(ALGraph &G,KeyType v,KeyType w)
//在图G中删除弧<v,w>, 成功返回OK, 否则返回ERROR
{
    // 请在这里补充代码, 完成本关任务
    /***** Begin *****/
    int m=LocateVex(G,v),n=LocateVex(G,w);
    int flag=0;
    if(m==-1||n==-1) return ERROR;
    ArcNode *p=G.vertices[m].firstarc;
    ArcNode *last=NULL;
    while(p!=NULL) {
        if(p->adjvex==n) {
            if(last==NULL) {
                G.vertices[m].firstarc=p->nextarc;
                free(p);
            }else{
                last->nextarc=p->nextarc;
                free(p);
            }
            flag=1;
            break;
        }else{
            last=p;
            p=p->nextarc;
        }
    }
    p=G.vertices[n].firstarc;
    last=NULL;
    while(p!=NULL) {
        if(p->adjvex==m) {
            if(last==NULL) {
                G.vertices[n].firstarc=p->nextarc;
                free(p);
            }else{
                last->nextarc=p->nextarc;
                free(p);
            }
            flag=1;
            break;
        }else{

```

```
        last=p;
        p=p->nextarc;
    }
}
if(flag==0) return ERROR;
G.arcnum--;
return OK;
/***** End *****/
}

void DFS(ALGraph G, int i, bool visited[], void
(*visit)(VertexType)) {
    visited[i] = true;
    visit(G.vertices[i].data);
    ArcNode *p = G.vertices[i].firstarc;
    while (p != NULL) {
        if (!visited[p->adjvex]) {
            DFS(G, p->adjvex, visited, visit);
        }
        p = p->nextarc;
    }
}

status DFSTraverse(ALGraph &G, KeyType startKey, void
(*visit)(VertexType)) {
    if (G.vexnum == 0) return ERROR;

    bool visited[MAX_VERTEX_NUM] = {false};

    if (startKey != -1) { // 如果指定了起始顶点
        int startIndex = LocateVex(G, startKey);
        if (startIndex == -1) {
            printf("起始顶点 '%d' 不存在! \n", startKey);
            return ERROR;
        }
        DFS(G, startIndex, visited, visit);
    } else { // 如果未指定起始顶点, 遍历整个图
        for (int i = 0; i < G.vexnum; i++) {
            if (!visited[i]) {
                DFS(G, i, visited, visit);
            }
        }
    }

    return OK;
}

status BFSTraverse(ALGraph &G, KeyType startKey, void
(*visit)(VertexType)) {
    if (G.vexnum == 0) return ERROR;

    bool visited[MAX_VERTEX_NUM] = {false};
    int QUEUE[MAX_VERTEX_NUM];
```

华中科技大学课程实验报告

```
int head = 0, tail = 0;

if (startKey != -1) { // 如果指定了起始顶点
    int startIndex = LocateVex(G, startKey);
    if (startIndex == -1) {
        printf("起始顶点 '%d' 不存在! \n", startKey);
        return ERROR;
    }
    visited[startIndex] = true;
    visit(G.vertices[startIndex].data);
    QUEUE[tail++] = startIndex;

    while (head != tail) {
        int current = QUEUE[head++];
        ArcNode *p = G.vertices[current].firstarc;
        while (p != NULL) {
            if (!visited[p->adjvex]) {
                visited[p->adjvex] = true;
                visit(G.vertices[p->adjvex].data);
                QUEUE[tail++] = p->adjvex;
            }
            p = p->nextarc;
        }
    }
} else { // 如果未指定起始顶点, 遍历整个图
    for (int v = 0; v < G.vexnum; v++) {
        if (!visited[v]) {
            visited[v] = true;
            visit(G.vertices[v].data);
            QUEUE[tail++] = v;

            while (head != tail) {
                int current = QUEUE[head++];
                ArcNode *p =
G.vertices[current].firstarc;
                while (p != NULL) {
                    if (!visited[p->adjvex]) {
                        visited[p->adjvex] = true;
                        visit(G.vertices[p-
>adjvex].data);

                        QUEUE[tail++] = p->adjvex;
                    }
                    p = p->nextarc;
                }
            }
        }
    }
}

return OK;
}
```

华中科技大学课程实验报告

```
std::vector<int> VerticesSetLessThanK(ALGraph &G, KeyType
v, int k) {
    std::vector<int> result; // 存储结果集合
    if (k <= 0) return result; // 如果 k <= 0, 直接返回空集合

    int start = LocateVex(G, v); // 查找顶点 v 的位置
    if (start == -1) return result; // 如果顶点 v 不存在, 返回空集合

    std::vector<bool> visited(G.vexnum, false); // 记录顶点
    是否访问过
    std::vector<int> distance(G.vexnum, 0); // 记录每个顶点
    到 v 的距离
    std::queue<int> q; // 队列用于广度优先搜索

    visited[start] = true;
    q.push(start);

    while (!q.empty()) {
        int current = q.front();
        q.pop();

        ArcNode *p = G.vertices[current].firstarc;
        while (p != NULL) {
            int adj = p->adjvex;
            if (!visited[adj]) {
                visited[adj] = true;
                distance[adj] = distance[current] + 1;
                if (distance[adj] < k) {

result.push_back(G.vertices[adj].data.key); // 将顶点关键字
加入结果集合

                    q.push(adj);
                }
            }
            p = p->nextarc;
        }
    }

    return result;
}

int ShortestPathLength(ALGraph &G, KeyType v, KeyType w) {
    int start = LocateVex(G, v); // 查找顶点 v 的位置
    int end = LocateVex(G, w); // 查找顶点 w 的位置

    if (start == -1 || end == -1) {
        return -1; // 如果顶点 v 或 w 不存在, 返回 -1 表示不存
        在路径
    }
}
```

```
if (start == end) {
    return 0; // 如果起点和终点相同，最短路径长度为 0
}

std::vector<bool> visited(G.vexnum, false); // 记录顶点
是否访问过
std::vector<int> distance(G.vexnum, 0); // 记录每个顶
点到起点的距离
std::queue<int> q; // 队列用于广
度优先搜索

visited[start] = true;
q.push(start);

while (!q.empty()) {
    int current = q.front();
    q.pop();

    ArcNode *p = G.vertices[current].firstarc;
    while (p != NULL) {
        int adj = p->adjvex;
        if (!visited[adj]) {
            visited[adj] = true;
            distance[adj] = distance[current] + 1;

            if (adj == end) {
                return distance[adj]; // 找到目标顶点，返
回最短路径长度
            }

            q.push(adj);
        }
        p = p->nextarc;
    }
}

return -1; // 如果无法到达目标顶点，返回 -1 表示不存在路径
}

int ConnectedComponentsNums(ALGraph &G) {
    if (G.vexnum == 0) {
        return 0; // 如果图中没有顶点，连通分量为 0
    }

    std::vector<bool> visited(G.vexnum, false); // 记录顶点
    是否访问过
    int componentCount = 0; // 连通分量计数

    // 深度优先搜索辅助函数
```

华中科技大学课程实验报告

```
auto DFS = [&](int v, auto &DFS) -> void {
    visited[v] = true;
    ArcNode *p = G.vertices[v].firstarc;
    while (p != NULL) {
        if (!visited[p->adjvex]) {
            DFS(p->adjvex, DFS);
        }
        p = p->nextarc;
    }
};

// 遍历所有顶点，查找未访问的顶点并进行 DFS
for (int i = 0; i < G.vexnum; i++) {
    if (!visited[i]) {
        componentCount++; // 每次发现一个未访问的顶点，连通
分量加 1
        DFS(i, DFS);
    }
}

return componentCount;
}

status SaveGraph(ALGraph G, char FileName[])

{
    FILE *fp = fopen(FileName, "w");
    if (fp == NULL) return ERROR;

    fprintf(fp, "%d %d\n", G.vexnum, G.arcnum);

    for (int i = 0; i < G.vexnum; i++) {
        fprintf(fp, "%d %s\n",
G.vertices[i].data.key, G.vertices[i].data.others);
    }

    for (int i = 0; i < G.vexnum; i++) {
        ArcNode *p = G.vertices[i].firstarc;
        while (p != NULL) {
            fprintf(fp, "%d %d\n", i, p->adjvex);
            p = p->nextarc;
        }
    }

    fclose(fp);
    return OK;
}

status LoadGraph(ALGraph &G, char FileName[])
```

```
{
    FILE *fp = fopen(FileName, "r");
    if (fp == NULL) return ERROR;

    fscanf(fp, "%d %d", &G.vexnum, &G.arcnum);

    for (int i = 0; i < G.vexnum; i++) {
        fscanf(fp, "%d %s", &G.vertices[i].data.key,
G.vertices[i].data.others);
        G.vertices[i].firstarc = NULL;
    }

    int from, to;
    while (fscanf(fp, "%d %d", &from, &to) != EOF)
    {
        ArcNode *newNode = (ArcNode
*)malloc(sizeof(ArcNode));
        newNode->adjvex = to;
        newNode->nextarc = NULL;

        if (G.vertices[from].firstarc == NULL) {

            G.vertices[from].firstarc = newNode;
        } else {

            ArcNode *tail = G.vertices[from].firstarc;
            while (tail->nextarc != NULL) {
                tail = tail->nextarc;
            }
            tail->nextarc = newNode;
        }
    }

    fclose(fp);
    return OK;
}
```